ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ ระบบวิเคราะห์ความคล้องจองของคำภาษาไทย
A System for Thai Rhyme Word Analysis

ผู้จัดทำ นาย อริญชัย แสงจันทร์

ปีการศึกษา 2567

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.กฤษณะ สีพนมวัน)

..... กรรมการสอบ
(อาจารย์เสฐลัทธ์ รอดเหตุภัย)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(อาจารย์ ดร.วัสรา รอดเหตุภัย)

ระบบวิเคราะห์ความคล้องจองของคำภาษาไทย

A System for Thai Rhyme Word Analysis

อริญชัย แสงจันทร์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2567

ชื่อปริญญานิพนธ์ ระบบวิเคราะห์ความคล้องจองของคำภาษาไทย
A System for Thai Rhyme Word Analysis
ผู้จัดทำ นาย อริญชัย แสงจันทร์
อาจารย์ที่ปรึกษา อาจารย์ ดร.วัสรา รอดเหตุภัย, อาจารย์เสฐลัทธ์ รอดเหตุภัย (ที่ปรึกษาร่วม)
ชื่อปริญญา วิทยาศาสตรบัณฑิต (วิทยาการคอมพิวเตอร์)
ปีการศึกษา 2567

บทคัดย่อ

โครงงานปริญญานิพนธ์นี้ มีวัตถุประสงค์เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับการตรวจสอบคำคล้องจองในภาษาไทย ซึ่งสามารถรองรับการใช้งานได้หลากหลายรูปแบบ ได้แก่ การตรวจสอบคำคล้องจองในกลอนสี่ กลอนหก และกลอนเจ็ด รวมถึงการเปรียบเทียบคำคล้องจองระหว่างคำ โดยผู้ใช้สามารถเข้าถึงสื่อการสอนเบื้องต้นเกี่ยวกับคำคล้องจองในภาษาไทย เพื่อพัฒนาทักษะการใช้คำคล้องจอง และสามารถตรวจสอบคำคล้องจองในแต่ละประเภทได้อย่างสะดวก นอกจากนี้เว็บแอปพลิเคชันยังรองรับการป้อนข้อมูลด้วยเสียงจากผู้ใช้งาน โดยระบบจะทำการแปลงเสียงจากผู้ใช้งานเป็นข้อความ เพื่อเพิ่มความสะดวกในการใช้งานและลดความยุ่งยากจากการพิมพ์ข้อความ ผู้ใช้ยังสามารถเล่นเกมเพื่อเสริมทักษะการใช้คำคล้องจอง ได้แก่ เกมต่อคำคล้องจองและเกมจับคู่คำคล้องจอง ซึ่งจะช่วยให้ผู้ใช้สามารถพัฒนาทักษะการใช้คำคล้องจองได้ดียิ่งขึ้น

คำสำคัญ : คำคล้องจอง, เว็บแอปพลิเคชัน, การตรวจสอบคำคล้องจอง, เกม, การแปลงเสียงเป็นข้อความ
Keyword : Rhyme word, Web application, Rhyme checking, Games, Speech-to-Text

กิตติกรรมประกาศ

ปริญญานิพนธ์นี้ สำเร็จลุล่วงได้ด้วยดีด้วยความช่วยเหลือจากผู้มีพระคุณหลายท่าน ผู้จัดทำปริญญานิพนธ์ขอขอบคุณ อาจารย์ ดร.วัสรา รอดเหตุภัย อาจารย์ที่ปรึกษาโครงงานปริญญานิพนธ์ อาจารย์เสฐลัทธ์ รอดเหตุภัย อาจารย์ที่ปรึกษาโครงงานปริญญานิพนธ์ร่วม และ ผู้ช่วยศาสตราจารย์ ดร.กฤษณะ สีพนมวัน ประธานกรรมการสอบ ที่ได้เสียสละเวลาทำงาน เพื่อให้คำปรึกษา คำแนะนำ และปรับปรุงแก้ไขจนกระทั่งโครงงานนี้สำเร็จลุล่วงได้ด้วยดี

สุดท้ายนี้ขอขอบคุณทุกท่านที่มีส่วนร่วมในการสนับสนุน และช่วยเหลือ ซึ่งทำให้การทำงานในครั้งนี้เป็นไปอย่างราบรื่น และประสบความสำเร็จ รวมทั้งครอบครัว เพื่อน ๆ ที่ให้กำลังใจ และอยู่เคียงข้างตลอดระยะเวลาที่ผ่านมา

อริญชัย แสงจันทร์

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... ฉ
สารบัญรูป ..... ช
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.4 อุปกรณ์และเครื่องมือที่ใช้ 2
1.4.1 ฮาร์ดแวร์ 2
1.4.2 ซอฟต์แวร์ 2
1.5 ผลที่คาดว่าจะได้รับ 2
บทที่ 2 ผลงานที่เกี่ยวข้อง 3
2.1 RHYMETHAI 3
2.1.1 ข้อเด่น 3
2.1.2 ข้อด้อย 3
2.2 Rhymer 5
2.2.1 ข้อเด่น 5
2.3 B – RHYMES 6
2.3.1 ข้อเด่น 6
2.3.2 ข้อด้อย 6
2.4 Rhymey 7
2.4.1 ข้อเด่น 7
2.4.2 ข้อด้อย 7
2.5 Shiritori 9
2.5.1 ข้อเด่น 9
2.5.2 ข้อด้อย 9
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 12
3.1 Speech to Text 12
3.2 Tailwind CSS 14
3.3 DaisyUI 14
3.4 Docker 15

3.5 PyThaiNLP 17
3.6 รูปสระในภาษาไทย 18
3.7 มาตราตัวสะกด 19
3.8 คำคล้องจอง 19
3.8.1 คำคล้องจอง 1 พยางค์ 19
3.8.2 คำคล้องจอง 2 พยางค์ 19
3.8.3 คำคล้องจอง 3 พยางค์ 19
3.8.4 คำคล้องจอง 4 พยางค์ 19
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 20
4.1 ขั้นตอนการดำเนินงาน 20
4.1.1 รวบรวมข้อมูลและกำหนดขอบเขตของงาน 20
4.1.2 ศึกษางานและทฤษฎีที่เกี่ยวข้อง 20
4.1.3 ออกแบบหน้าเว็บแอปพลิเคชันและระบบงาน 20
4.1.4 พัฒนาโปรแกรม 20
4.1.5 ทดสอบและแก้ไขข้อผิดพลาด 20
4.1.6 จัดทำเอกสารประกอบโครงงาน 20
4.2 แผนการดำเนินงาน 21
บทที่ 5 การวิเคราะห์และออกแบบระบบ 22
5.1 สถาปัตยกรรมของระบบ 22
5.2 การทำงานของระบบตรวจสอบคำคล้องจอง 23
5.2.1 การทำงานของระบบตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ 24
5.2.2 การทำงานของระบบตรวจสอบกลอน 26
5.3 การทำงานเกมจับคู่คำคล้องจอง 34
5.3.1 รายละเอียดการเลือกระดับความยาก 35
5.4 การทำงานของเกมต่อคำคล้องจอง 36
5.5 การทำงานของระบบแปลงเสียงเป็นข้อความ 38
5.6 ส่วนของฐานข้อมูล 38
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 39
6.1 ผลการดำเนินงาน 39
6.1.1 สื่อการเรียนการสอนเบื้องต้นเกี่ยวกับคำคล้องจองภาษาไทย 39
6.1.2 ตรวจสอบคำคล้องจอง 40
6.1.3 เกมจับคู่คำคล้องจอง 41
6.1.4 เกมต่อคำคล้องจอง 42
6.1.5 การแปลงข้อมูลเสียงเป็นข้อความ 43
6.2 ข้อจำกัด 43
6.3 ข้อเสนอแนะ 44
บรรณานุกรม 45

ภาคผนวก ก คู่มือการใช้งาน 46
ก.1 หน้าแรกของเว็บแอปพลิเคชัน 46
ก.2 หน้าเกม 46
ก.2.1 เกมจับคู่คำคล้องจอง 47
ก.2.2 เกมต่อคำคล้องจอง 48
ก.3 หน้าเรียนรู้คำคล้องจอง 49
ก.4 หน้าตรวจคำคล้องจอง 50
ก.4.1 เปรียบเทียบคำคล้องจอง 50
ก.4.2 ตรวจสอบกลอนสี่ 51
ก.4.3 ตรวจสอบกลอนหก 51
ก.4.4 ตรวจสอบกลอนเจ็ด 52
ก.5 หน้า About 52

สารบัญตาราง

หน้า

ตารางที่ 3.1 สระในภาษาไทย 18
ตารางที่ 3.2 มาตราตัวสะกด 19
ตารางที่ 4.1 แผนการดำเนินงาน 21

สารบัญรูป

หน้า

รูปที่ 2.1 หน้าค้นหาคำคล้องจองของ RHYMETHAI 4
รูปที่ 2.2 หน้าคำที่ค้นหาของ RHYMETHAI 4
รูปที่ 2.3 วิธีการค้นหาของ Rhymer 5
รูปที่ 2.4 หน้าคำที่ค้นหาของ Rhymer 5
รูปที่ 2.5 หน้าค้นหาคำคล้องจองของ B-RHYMES 6
รูปที่ 2.6 หน้าคำที่ค้นหาของ B-RHYMES 7
รูปที่ 2.7 ผลลัพธ์การค้นหาของ Rhymey แสดงคำคล้องจอง 8
รูปที่ 2.8 ผลลัพธ์การค้นหาของ Rhymey แสดงคำคล้องจองใกล้เคียง 8
รูปที่ 2.9 หน้าแรกเมื่อเข้าเว็บไซต์ Shiritori 10
รูปที่ 2.10 หน้า Lobby โหมด Multiplayer ของเว็บไซต์ Shiritori 10
รูปที่ 2.11 ระดับการเล่น โหมด Single Player ของเว็บไซต์ Shiritori 11
รูปที่ 2.12 หน้าการเล่นเกมของเว็บไซต์ Shiritori 11
รูปที่ 3.1 ตัวอย่างกระบวนการแปลงเสียงเป็นข้อความ 12
รูปที่ 3.2 ตัวอย่างคำสั่งติดตั้ง Speech Recognition 12
รูปที่ 3.3 ตัวอย่างโค้ดระบบแปลงเสียงเป็นข้อความ 13
รูปที่ 3.4 ตัวอย่างคำสั่งสร้างปุ่มด้วย Tailwind CSS 14
รูปที่ 3.5 ตัวอย่างผลลัพธ์การสร้างปุ่มด้วย Tailwind CSS 14
รูปที่ 3.6 ตัวอย่างคำสั่งสร้างปุ่มด้วย DaisyUI 15
รูปที่ 3.7 ตัวอย่างผลลัพธ์การสร้างปุ่มด้วย DaisyUI 15
รูปที่ 3.8 Docker 16
รูปที่ 3.9 ตัวอย่างการเช็คเวอร์ชันของ Docker 16
รูปที่ 3.10 ตัวอย่างทดสอบการใช้งาน Docker 16
รูปที่ 3.11 PyThaiNLP 17
รูปที่ 3.12 ตัวอย่างการตัดพยางค์โดยการใช้ Syllable_Tokenize 17
รูปที่ 3.13 ตัวอย่างผลลัพธ์การตัดพยางค์โดยการใช้ Syllable_Tokenize 17
รูปที่ 3.14 ตัวอย่างการตรวจสอบคำคล้องจองโดยการใช้ KhaveeVerifier 18
รูปที่ 3.15 ตัวอย่างผลลัพธ์การตรวจสอบคำคล้องจองโดยการใช้ KhaveeVerifier 18
รูปที่ 5.1 สถาปัตยกรรมระบบ 22
รูปที่ 5.2 Flowchart การทำงานของระบบตรวจสอบคำคล้องจอง 23
รูปที่ 5.3 โค้ดตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ (1) 24
รูปที่ 5.4 โค้ดตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ (2) 25
รูปที่ 5.5 โค้ดตรวจสอบกลอน 26
รูปที่ 5.6 โค้ดแยกบทกลอน 27

รูปที่ 5.7 โค้ดแยกวรรคและพยางค์ 27
รูปที่ 5.8 โค้ดตรวจสอบจำนวนวรรคในแต่ละบท 27
รูปที่ 5.9 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนสี่ 27
รูปที่ 5.10 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 1 28
รูปที่ 5.11 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 2 28
รูปที่ 5.12 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 3 28
รูปที่ 5.13 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 4 29
รูปที่ 5.14 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนหก 30
รูปที่ 5.15 โค้ดตรวจสอบกลอนหก กฎข้อที่ 1 30
รูปที่ 5.16 โค้ดตรวจสอบกลอนหก กฎข้อที่ 2 30
รูปที่ 5.17 โค้ดตรวจสอบกลอนหก กฎข้อที่ 3 31
รูปที่ 5.18 โค้ดตรวจสอบกลอนหก กฎข้อที่ 4 31
รูปที่ 5.19 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนเจ็ด 32
รูปที่ 5.20 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 1 33
รูปที่ 5.21 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 2 33
รูปที่ 5.22 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 3 33
รูปที่ 5.23 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 4 33
รูปที่ 5.24 Flowchart การทำงานของเกมจับคู่คำคล้องจอง 34
รูปที่ 5.25 Flowchart การทำงานของเกมต่อคำคล้องจอง 37
รูปที่ 5.26 ตัวอย่างชุดข้อมูลคำศัพท์ชั้นประถมศึกษาปีที่ 1 38
รูปที่ 6.1 สื่อการเรียนการสอนเบื้องต้นเกี่ยวกับคำคล้องจองภาษาไทย 39
รูปที่ 6.2 ตัวอย่างการเปรียบเทียบคำคล้องจองระหว่างคำและผลลัพธ์ 40
รูปที่ 6.3 ตัวอย่างการตรวจสอบคำคล้องจองในกลอนทุกประเภทและผลลัพธ์ 40
รูปที่ 6.4 ระดับความยากเกมจับคู่คำคล้องจอง 41
รูปที่ 6.5 ตัวอย่างคำถามและคำตอบเกมจับคู่คำคล้องจอง 41
รูปที่ 6.6 ตัวอย่างผลลัพธ์เกมจับคู่คำคล้องจองเมื่อตอบคำถามครบทุกข้อ 42
รูปที่ 6.7 ตัวอย่างคำตั้งต้นเกมต่อคำคล้องจอง 42
รูปที่ 6.8 ตัวอย่างผลลัพธ์เกมต่อคำคล้องจองเมื่อคำตอบไม่คล้องจองกับคำถาม 43
รูปที่ 6.9 ตัวอย่างระบบแปลงเสียงเป็นข้อความ 43
รูปผนวกที่ ก.1 หน้าแรกของเว็บแอปพลิเคชัน 46
รูปผนวกที่ ก.2 หน้าเลือกเกมของเว็บแอปพลิเคชัน 46
รูปผนวกที่ ก.3 หน้าเลือกระดับความยากของเกมจับคู่คำคล้องจอง 47
รูปผนวกที่ ก.4 หน้าเกมจับคู่คำคล้องจองขณะเล่นเกม 47
รูปผนวกที่ ก.5 หน้าแสดงผลลัพธ์หลังจบเกมจับคู่คำคล้องจอง 48
รูปผนวกที่ ก.6 หน้าเกมต่อคำคล้องจองขณะเล่นเกม 48
รูปผนวกที่ ก.7 หน้าแสดงผลลัพธ์หลังจบเกมต่อคำคล้องจอง 49
รูปผนวกที่ ก.8 หน้าสื่อการสอนของเว็บไซต์ 49

รูปผนวกที่ ก.9 ตรวจสอบคำคล้องจอง (เปรียบเทียบระหว่างคำ) 50
รูปผนวกที่ ก.10 ตรวจสอบกลอนสี่สุภาพ 51
รูปผนวกที่ ก.11 ตรวจสอบกลอนหกสุภาพ 51
รูปผนวกที่ ก.12 ตรวจสอบกลอนเจ็ดสุภาพ 52
รูปผนวกที่ ก.13 หน้า About 52

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ในยุคปัจจุบัน เว็บแอปพลิเคชันได้เข้ามามีบทบาทสำคัญต่อการศึกษามากยิ่งขึ้น โดยเปิดโอกาสให้ผู้ใช้สามารถเข้าถึงแหล่งข้อมูลเพื่อการศึกษาได้อย่างหลากหลาย ส่งผลให้รูปแบบการเรียนการสอนในชั้นเรียนต้องมีการเปลี่ยนแปลงไปตามยุคสมัย เพื่อให้สอดคล้องกับความก้าวหน้าทางเทคโนโลยี และความต้องการของผู้ใช้ในปัจจุบัน

คำคล้องจองจัดเป็นอีกหนึ่งลักษณะเด่นของภาษาไทย อันเนื่องมาจากในภาษาไทยมีการใช้คำเหล่านี้เพื่อใช้สร้างความสนุกสนาน ความบันเทิง และการสื่อสารในการเล่าเรื่องต่าง ๆ สำหรับการเรียนรู้เรื่องคำคล้องจองได้มีการเริ่มต้นการเรียนการสอนตั้งแต่ระดับชั้นประถมศึกษาปีที่ 1 โดยมีจุดประสงค์เพื่อให้ได้ฝึกอ่านคำคล้องจอง ฝึกกระบวนการคิด และการออกเสียงคำให้ชัดเจน

ปัจจุบันการศึกษาคำคล้องจองยังประสบปัญหาอยู่หลายประการที่ส่งผลต่อการเรียนรู้ของผู้เรียนในระดับชั้นประถมศึกษาหนึ่งในปัจจัยที่สำคัญคือผู้เรียนยังไม่เข้าใจการใช้คำคล้องจองได้อย่างถ่องแท้ เนื่องจากสื่อการเรียนการสอนในชั้นเรียนและตัวอย่างการใช้คำคล้องจองอาจไม่เพียงพอ อีกทั้งในการสอนแบบดั้งเดิมอาจไม่น่าสนใจหรือไม่สามารถดึงดูดความสนใจของผู้เรียนได้ดีพอ

ด้วยเหตุนี้ผู้วิจัยมีความเห็นว่าการพัฒนาเว็บแอปพลิเคชันคำคล้องจองสำหรับระดับประถมศึกษาจะช่วยให้ผู้เรียนสามารถเข้าถึงแหล่งเรียนรู้คำคล้องจองได้หลากหลายมากยิ่งขึ้น ผ่านเกมการศึกษา และสื่อการสอนต่าง ๆ ซึ่งจะสร้างความน่าสนใจและทำให้ตัวผู้เรียนสามารถพัฒนาทักษะการใช้คำคล้องจองได้ดียิ่งขึ้น

1.2 วัตถุประสงค์

1) เพื่อพัฒนาเว็บแอปพลิเคชันที่ช่วยให้ผู้ใช้งานสามารถเรียนรู้และทำความเข้าใจเกี่ยวกับคำคล้องจองในภาษาไทย
2) เพื่อพัฒนาเครื่องมือที่ใช้ในการตรวจสอบคำคล้องจองในรูปแบบต่าง ๆ เช่น กลอนสี่ กลอนหก กลอนเจ็ด และเปรียบเทียบคำคล้องจองระหว่างคำ

1.3 ลักษณะและขอบเขต

โครงงานวิจัยนี้มีการพัฒนาเว็บแอปพลิเคชันเพื่อส่งเสริมการศึกษาเกี่ยวกับคำคล้องจองภาษาไทย โดยจะมีลักษณะและขอบเขตการทำงานของเว็บแอปพลิเคชัน ดังนี้

1) ผู้ใช้สามารถเข้าถึงสื่อการสอนเบื้องต้นเกี่ยวกับคำคล้องจองในภาษาไทย
2) ผู้ใช้สามารถตรวจสอบคำคล้องจองในรูปแบบต่าง ๆ ได้แก่ กลอนสี่ กลอนหก กลอนเจ็ด และสามารถเปรียบเทียบคำคล้องจองระหว่างคำได้
3) เว็บแอปพลิเคชันสามารถรองรับการป้อนข้อมูลด้วยเสียงจากผู้ใช้งานและแปลงเสียงเป็นข้อความได้
4) ผู้ใช้สามารถเล่นเกมเพื่อเสริมทักษะการใช้คำคล้องจอง ได้แก่
   - เกมต่อคำคล้องจอง: ในเกมนี้ผู้เล่นต้องตอบคำที่คล้องจองกับคำที่ระบบแสดงออกมา โดยจะต้องตอบคำที่คล้องจองไปเรื่อย ๆ จนกว่าจะตอบผิด เมื่อผู้เล่นตอบผิดเกมจะหยุดและแสดงผลลัพธ์ทันที
   - เกมจับคู่คำคล้องจอง: ในเกมนี้ผู้เล่นจะต้องเลือกจับคู่คำตอบที่คล้องจองกับคำถามให้ถูกต้อง โดยผู้เล่นต้องตอบให้ครบทุกข้อ เมื่อผู้เล่นตอบครบทุกข้อเกมจะหยุดและแสดงผลลัพธ์ทันที

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

คอมพิวเตอร์ตั้งโต๊ะ (Desktop)

- หน่วยประมวลผลกลาง Intel Core i5 7400
- หน่วยประมวลผลกราฟิก NVIDIA GeForce GTX 1060
- หน่วยความจำหลัก 16.00 GB

โน้ตบุ๊ค (Laptop)

- หน่วยประมวลผลกลาง Intel Core i7 9750H
- หน่วยประมวลผลกราฟิก NVIDIA GeForce GTX 1650
- หน่วยความจำหลัก 16.00 GB

1.4.2 ซอฟต์แวร์

- ภาษาและไลบรารี่ที่ใช้ในการพัฒนา : React.js, Python, Tailwind CSS, DaisyUI
- โปรแกรมและเครื่องมือที่ใช้ในการพัฒนา : Visual Studio Code, Figma, Docker, phpMyAdmin, MySQL

1.5 ผลที่คาดว่าจะได้รับ

1) ผู้ใช้มีทักษะในการใช้คำคล้องจองได้อย่างคล่องแคล่ว และถูกต้องมากยิ่งขึ้น
2) ผู้ใช้สามารถนำคำคล้องจองไปประยุกต์ใช้ในสถานการณ์ต่าง ๆ รวมถึงการแต่งกลอน และบทกวีได้อย่างถูกต้อง
3) ผู้ใช้ได้รับความสะดวกในการตรวจสอบคำคล้องจองผ่านเว็บแอปพลิเคชัน

บทที่ 2 ผลงานที่เกี่ยวข้อง

ในบทที่ 2 นี้จะอธิบายถึงผลงานที่เกี่ยวข้องกับโครงงาน เพื่อทำการวิเคราะห์ความสัมพันธ์และปัญหาที่เกี่ยวข้อง โดยจะมีการสรุปข้อดีและข้อเสียของแต่ละผลงานวิจัยเพื่อใช้เป็นข้อมูลในการปรับปรุงและพัฒนาโครงงานวิจัยต่อไป

2.1 RHYMETHAI

RHYMETHAI [1] เป็นเว็บแอปพลิเคชันสำหรับค้นหาคำคล้องจองภาษาไทยแบบอัตโนมัติ ช่วยให้ผู้ใช้งานสามารถค้นหาคำที่คล้องจองเพื่อนำไปใช้แต่งเพลงหรือสร้างสรรค์ผลงานได้อย่างสะดวกและรวดเร็ว (ดังแสดงในรูปที่ 2.1) ระบบทำงานโดยการรับคำที่ต้องการค้นหาจากผู้ใช้งาน และทำการแสดงผลการค้นหาคำที่คล้องจองออกมาเป็น 4 หมวดหมู่ (ดังแสดงในรูปที่ 2.2) ได้แก่

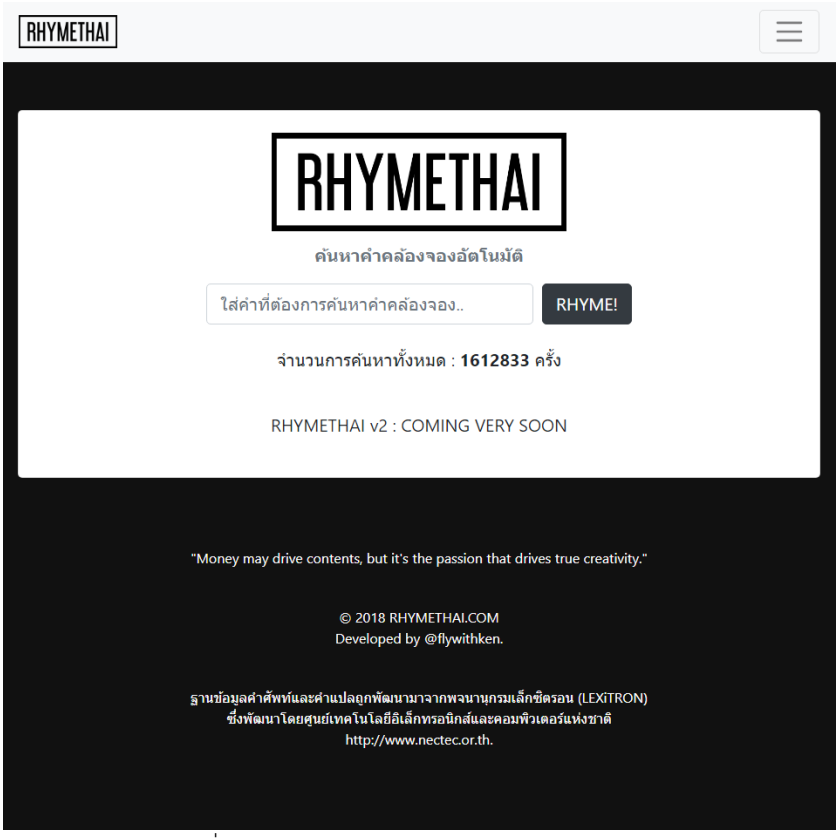
- คำที่คล้องจองโดยสมบูรณ์
- คำที่คล้องจองโดย “สระและตัวสะกด”
- คำที่คล้องจองโดย “สระและวรรณยุกต์”
- คำที่คล้องจอง “เฉพาะพยางค์สุดท้าย”

2.1.1 ข้อเด่น

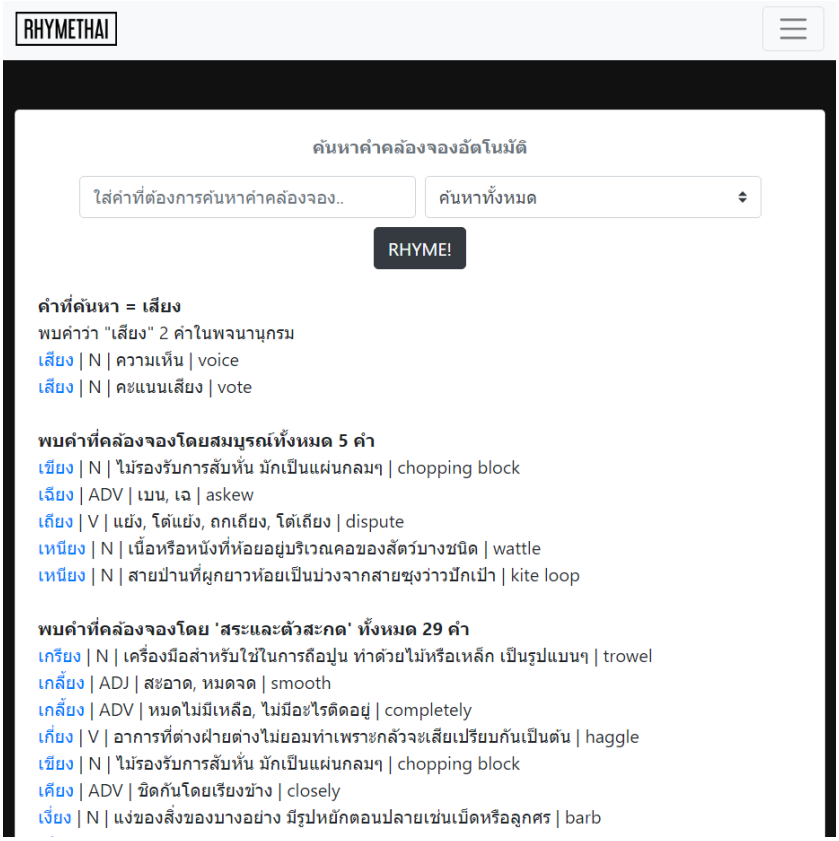
- สามารถบอก Noun / Verb / Adj. / Adv ของแต่ละคำได้
- สามารถบอกความหมายของคำ และชื่อภาษาอังกฤษของแต่ละคำได้
- มีการจัดหมวดหมู่คำชัดเจน

2.1.2 ข้อด้อย

- การจัดเรียงข้อความไม่เรียบร้อยทำให้อ่านยาก
- ตัวเลือกในการค้นหาคำคล้องจองมีให้เลือกแค่วิธีเดียว



รูปที่ 2.1 หน้าค้นหาคำคล้องจองของ RHYMETHAI



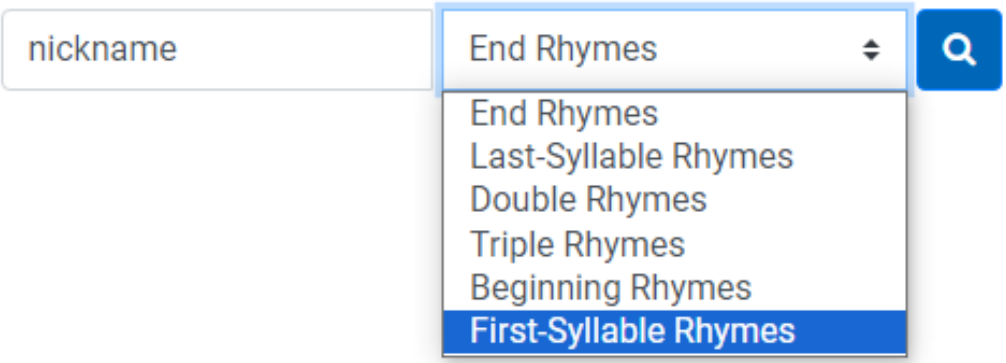
รูปที่ 2.2 หน้าคำที่ค้นหาของ RHYMETHAI

2.2 Rhymer

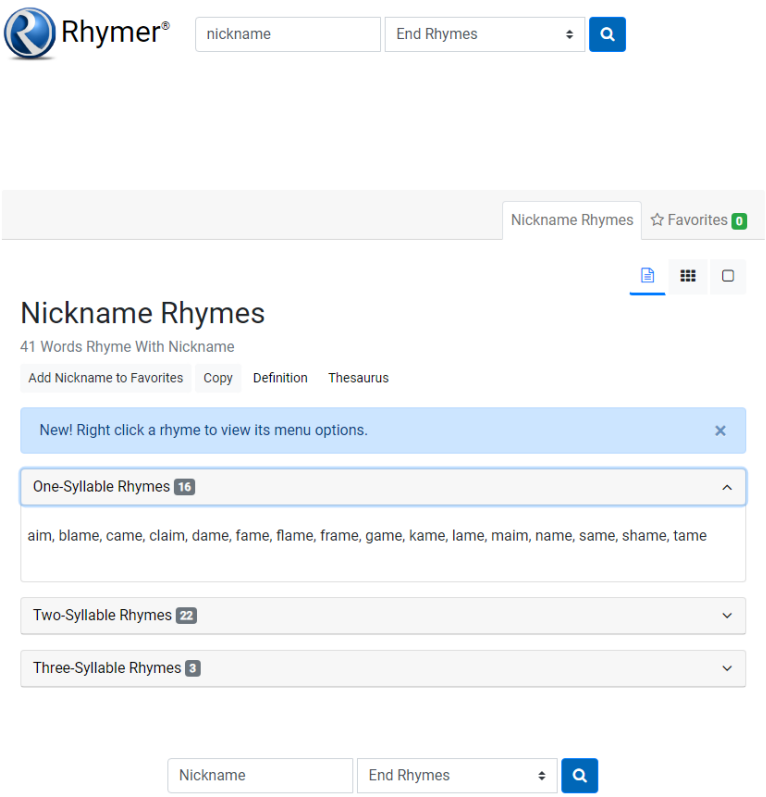
Rhymer [2] เป็นเว็บแอปพลิเคชันที่มีสื่อการสอนคำคล้องจอง พร้อมระบบค้นหาคำคล้องจองภาษาอังกฤษ (ดังแสดงในรูปที่ 2.3) โดยผู้ใช้งานสามารถเลือกวิธีการค้นหาได้หลายแบบ เช่น การค้นหาคำคล้องจองด้วยวิธี Fist-Syllable Rhymes ระบบจะทำการรับคำที่ต้องการค้นหาจากผู้ใช้งาน และเมื่อทำการค้นหาแล้ว ระบบจะแสดงผลลัพธ์ออกเป็นหมวดหมู่ (ดังแสดงในรูปที่ 2.4)

2.2.1 ข้อเด่น

- สามารถเลือกวิธีการตรวจสอบหรือค้นหาได้หลากหลาย
- UI ใช้งานง่าย



รูปที่ 2.3 วิธีการค้นหาของ Rhymer



รูปที่ 2.4 หน้าคำที่ค้นหาของ Rhymer

2.3 B – RHYMES

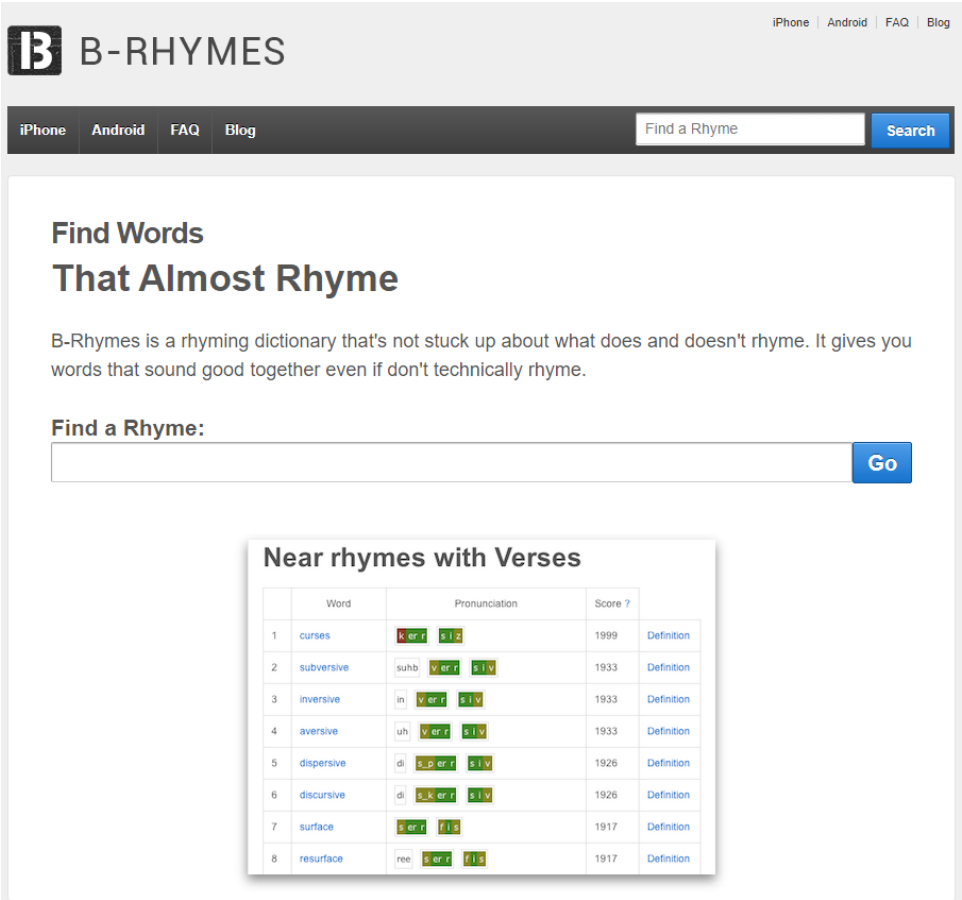
B – RHYMES [3] เป็นระบบค้นหาคำคล้องจองภาษาอังกฤษ (ดังแสดงในรูปที่ 2.5) ระบบจะทำการรับคำที่ต้องการค้นหาจากผู้ใช้งาน และเมื่อทำการค้นหาแล้วจะทำการแสดงข้อมูล การออกเสียง และคะแนน ของคำที่คล้องจองกับคำที่ค้นหา (ดังแสดงในรูปที่ 2.6)

2.3.1 ข้อเด่น

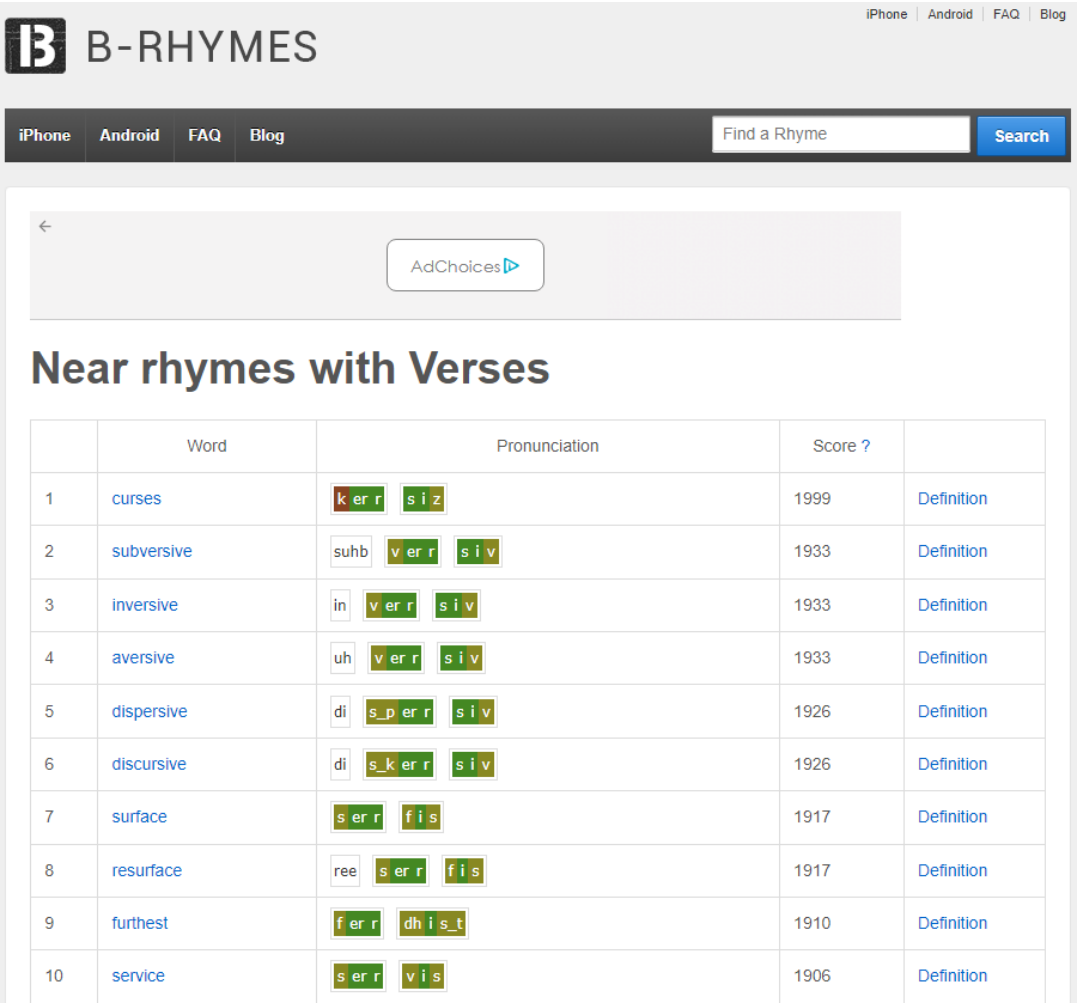
- การจัดเรียงข้อมูลมีความเป็นระเบียบเรียบร้อย
- มีการบอกการออกเสียงของคำแต่ละคำ
- คะแนนสามารถบอกถึงความคล้องจองของคำว่ามีความคล้องจองมากน้อยแค่ไหน ยิ่งคะแนนมากยิ่งมีความคล้องจองกับคำที่ค้นหามาก

2.3.2 ข้อด้อย

- คำที่แนะนำในระบบส่วนใหญ่เป็นคำที่มีเสียงใกล้เคียง (Near Rhymes) ซึ่งอาจไม่คล้องจองสมบูรณ์แบบตามที่ต้องการ จึงอาจไม่เหมาะสมสำหรับการใช้งานในกรณีที่ต้องการความแม่นยำในการคล้องจองเสียง เช่น การแต่งกลอนหรือเพลงที่ต้องการความถูกต้องและความสมบูรณ์แบบในการคล้องจองเสียง



รูปที่ 2.5 หน้าค้นหาคำคล้องจองของ B-RHYMES



รูปที่ 2.6 หน้าคำที่ค้นหาของ B-RHYMES

2.4 Rhymey

Rhymey [4] เป็น Extension ที่อยู่บน Chrome Web Store ช่วยให้ผู้ใช้สามารถหาคำคล้องจองภาษาอังกฤษในเว็บไซต์ด้วยการดับเบิลคลิกที่คำที่ต้องการหาคำที่คล้องจอง หลังการค้นหาจะแสดงคำที่คล้องจอง (ดังแสดงในรูปที่ 2.7) และคำคล้องจองที่ใกล้เคียง (Near Rhymes) (ดังแสดงในรูปที่ 2.8)

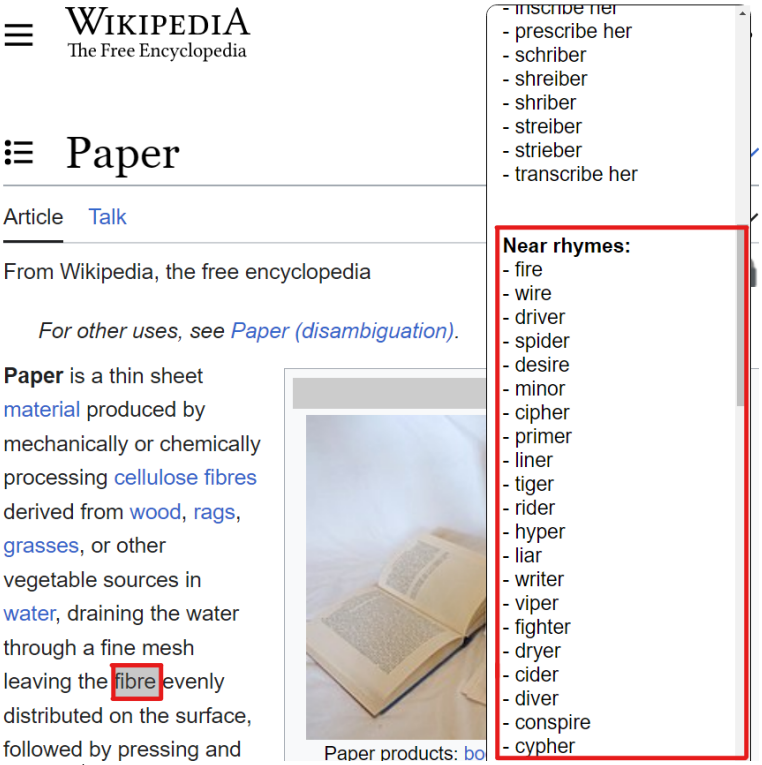
2.4.1 ข้อเด่น

- ใช้งานง่ายค้นหาคำคล้องจองได้ทันทีเมื่อเจอคำที่ต้องการค้นหา
- สามารถบอกคำที่คล้องจอง และคำคล้องจองที่ใกล้เคียง

2.4.2 ข้อด้อย

- คำที่เลือกจะต้องมีความยาวอย่างน้อย 3 ตัวอักษร
- ไม่สามารถปรับแต่ง หรือเคลื่อนย้ายหน้าต่างแสดงผลได้

รูปที่ 2.7 ผลลัพธ์การค้นหาของ Rhymey แสดงคำคล้องจอง



รูปที่ 2.8 ผลลัพธ์การค้นหาของ Rhymey แสดงคำคล้องจองใกล้เคียง

2.5 Shiritori

Shiritori [5] ในเวอร์ชันนี้จะเป็นเกมที่เล่นโดยใช้คำศัพท์ภาษาอังกฤษ ในหน้าแรกของเว็บไซต์จะแสดงส่วนต่าง ๆ เช่น ปุ่มเลือกเล่นคนเดียวและปุ่มเล่นหลายคน (ดังแสดงในรูปที่ 2.9) ในโหมดเล่นหลายคน ผู้เล่นสามารถเข้าไปในล็อบบี้เพื่อเลือกเล่นเกมร่วมกับผู้เล่นอื่น (ดังแสดงในรูปที่ 2.10) สำหรับโหมดเล่นคนเดียวจะมีระดับความยากอยู่ 4 ระดับ (ดังแสดงในรูปที่ 2.11) และเมื่อเข้าสู่หน้าการเล่นเกม ผู้เล่นจะเห็นการจับเวลาและคะแนนระหว่างการเล่น (ดังแสดงในรูปที่ 2.12)

ในการเล่นผู้เล่นคนที่หนึ่งจะได้รับตัวอักษรแบบสุ่ม และจะต้องพิมพ์คำศัพท์ที่ขึ้นต้นด้วยตัวอักษรนั้น หลังจากนั้นผู้เล่นคนที่สองจะต้องพิมพ์คำศัพท์ที่ขึ้นต้นด้วยอักษรตัวสุดท้ายของผู้เล่นคนแรกผลัดกันวนไปเรื่อย ๆ โดยตัวเกมจะมีคะแนนให้ผู้เล่นคนละ 100 คะแนน ผู้เล่นคนแรกที่ทำคะแนนได้จนเหลือศูนย์จะเป็นผู้ชนะ โดยวิธีการที่จะทำให้คะแนนลดมี 2 วิธี ดังนี้

1) โบนัสความยาว: เมื่อผู้เล่นพิมพ์คำศัพท์ที่มีตัวอักษรเกิน 4 ตัว จะได้รับคะแนนพิเศษเท่ากับจำนวนตัวอักษรที่เกิน เช่นคำว่า "apple" (5 ตัวอักษร) จะได้โบนัส 1 คะแนน (5 – 4 = 1)
2) โบนัสความเร็ว: ในแต่ละรอบ ตัวจับเวลาจะเริ่มนับถอยหลังจากเวลาตั้งต้น 15 วินาที หากผู้เล่นตอบก่อนเวลาหมด จะได้รับโบนัสเท่ากับเวลาที่เหลือ ตัวอย่างเช่น: เวลาตั้งต้น 15 วินาที หากตอบในเวลา 5 วินาที จะได้โบนัส 10 คะแนน (15 - 5 = 10)

ตัวอย่างการคำนวณคะแนน

1) ผู้เล่นพิมพ์คำว่า “orange” (6 ตัวอักษร) ได้โบนัสความยาว 2 คะแนน (6 - 4 = 2)
2) ผู้เล่นตอบในเวลา 8 วินาที จากเวลาตั้งต้น 15 วินาที ได้โบนัสความเร็ว 7 คะแนน (15 - 8 = 7)
3) รวมโบนัสทั้งสองประเภทคือ 9 คะแนน (2 + 7)
4) คะแนนของผู้เล่น 100 คะแนน จะลดลง 9 คะแนน ส่งผลให้ผู้เล่นเหลือ 91 คะแนน

กรณีโบนัสรวมติดลบ

หากผลรวมของโบนัสความยาวและโบนัสความเร็วติดลบ โบนัสจะถูกปรับให้เป็น 0 เช่น

1) ผู้เล่นพิมพ์คำว่า “orange” (6 ตัวอักษร) ได้โบนัสความยาว 2 คะแนน (6 - 4 = 2)
2) ผู้เล่นตอบในเวลา 18 วินาที จากเวลาตั้งต้น 15 วินาที ได้โบนัสความเร็ว -3 คะแนน (15 – 18 = -3)
3) รวมโบนัสทั้งสองประเภทคือ -1 คะแนน (2 - 3) แต่คะแนนจะถูกปรับเป็นศูนย์เนื่องจากคะแนนติดลบ
4) คะแนนของผู้เล่น 100 คะแนน จะไม่ลดลงในรอบนี้

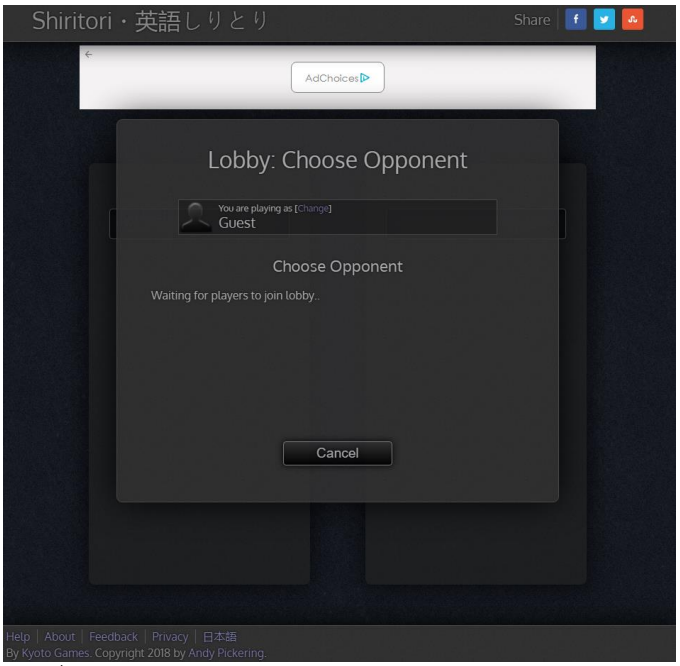
2.5.1 ข้อเด่น

- มีโหมดการเล่น 2 แบบ คือ Single Player และ Multiplayer
- โหมด Single Player มีการเล่น 4 ระดับ คือ Easy Medium Hard และ Extreme
- มีการจับเวลา และคะแนนทำให้เกมมีความท้าทายผู้เล่น

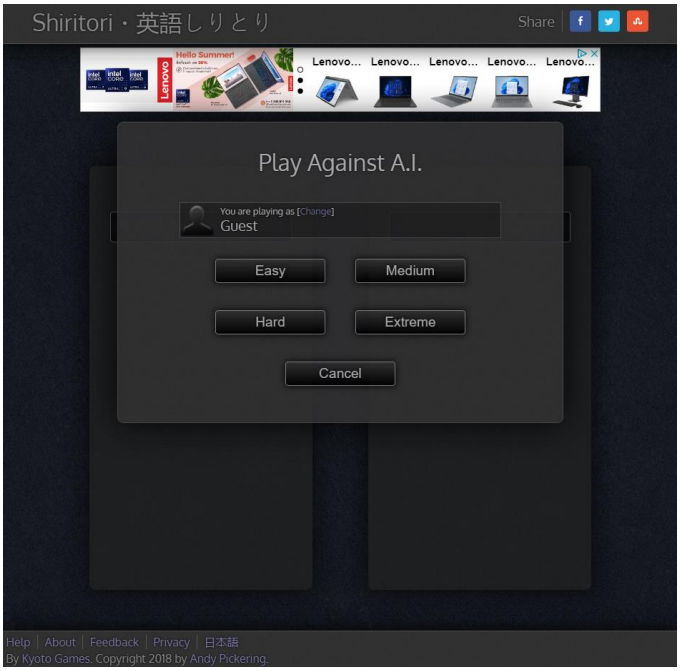
2.5.2 ข้อด้อย

- ไม่มีการอธิบายวิธีการเล่นอย่างละเอียด ทำให้ผู้เล่นใหม่ทำความเข้าใจได้ยาก

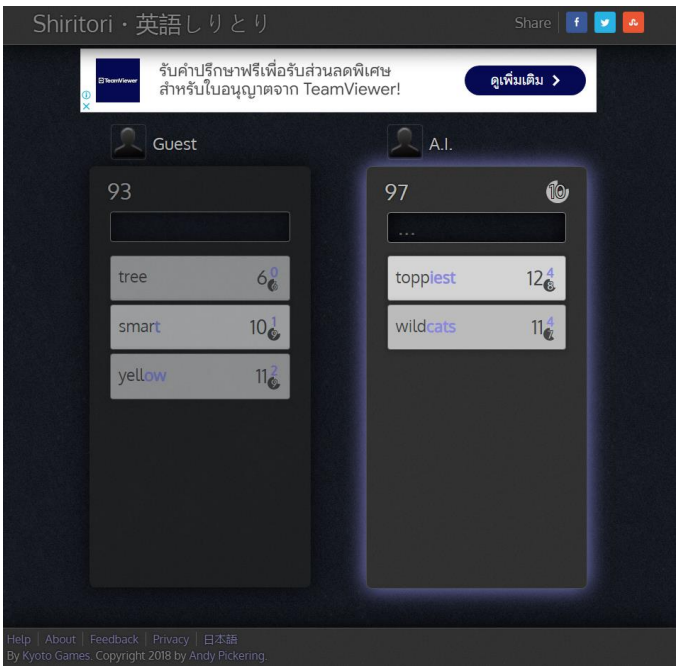
รูปที่ 2.9 หน้าแรกเมื่อเข้าเว็บไซต์ Shiritori



รูปที่ 2.10 หน้า Lobby โหมด Multiplayer ของเว็บไซต์ Shiritori



รูปที่ 2.11 ระดับการเล่น โหมด Single Player ของเว็บไซต์ Shiritori



รูปที่ 2.12 หน้าการเล่นเกมของเว็บไซต์ Shiritori

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

ในบทที่ 3 นี้จะนำเสนอเนื้อหาทางทฤษฎี และองค์ความรู้ที่เกี่ยวข้องกับการพัฒนาโครงงาน เพื่อให้ผู้อ่านเข้าใจถึงแนวคิดพื้นฐานและกรอบการพัฒนาที่ใช้เป็นแนวทางในการสร้างโครงงาน โดยมีรายละเอียดในแต่ละเนื้อหาดังต่อไปนี้

3.1 Speech to Text

การแปลงข้อมูลเสียงให้กลายเป็นข้อความ (Speech to Text) [6] เป็นเทคโนโลยีที่ช่วยในการแปลงเสียงของผู้ใช้งานให้กลายเป็นข้อความโดยอัตโนมัติ (ดังแสดงในรูปที่ 3.1) โดยเทคโนโลยีนี้สามารถประยุกต์ใช้ได้ในหลากหลายสถานการณ์ เช่น การพิมพ์ข้อความด้วยเสียง การสั่งงานผู้ช่วยอัจฉริยะ การแปลภาษาแบบเรียลไทม์ หรือแม้กระทั่งการจดบันทึกการประชุมโดยอัตโนมัติ ซึ่งช่วยเพิ่มความสะดวกสบายและลดเวลาในการทำงาน ทั้งยังสามารถรองรับหลายภาษาและสำเนียงที่แตกต่างกันได้อย่างมีประสิทธิภาพ

รูปที่ 3.1 ตัวอย่างกระบวนการแปลงเสียงเป็นข้อความ

สำหรับขั้นตอนในการติดตั้งและใช้งานระบบแปลงเสียงเป็นข้อความใน React.js สามารถทำได้ดังนี้

1) ทำการติดตั้งไลบรารีตามคำสั่ง (ดังแสดงในรูปที่ 3.2)
2) หลังจากติดตั้งสำเร็จ ให้ทำการนำโค้ดตัวอย่าง (ดังแสดงในรูปที่ 3.3) ไปใส่ในไฟล์ React Component

หลังจากนำโค้ดตัวอย่างในข้อที่สองไปใส่ โค้ดนี้จะทำการสร้าง Interface สำหรับการบันทึกเสียงด้วยปุ่มควบคุมการเริ่มต้น, หยุด และรีเซ็ตการแปลงเสียงเป็นข้อความ เมื่อผู้ใช้กดปุ่ม Start ระบบจะเริ่มบันทึกเสียงและแปลงเสียงเป็นข้อความ โดยจะทำการแสดงผลในตัวแปร transcript บนหน้าจอ หากผู้ใช้ต้องการหยุดบันทึกเสียง สามารถกดปุ่ม Stop และหากต้องการล้างข้อความ สามารถกดปุ่ม Reset

```
npm install --save react-speech-recognition
```

รูปที่ 3.2 ตัวอย่างคำสั่งติดตั้ง Speech Recognition

```
import React from 'react';
import SpeechRecognition, { useSpeechRecognition } from 'react-speech-recognition';

const Dictaphone = () => {
  const {
    transcript,
    listening,
    resetTranscript,
    browserSupportsSpeechRecognition
  } = useSpeechRecognition();

  if (!browserSupportsSpeechRecognition) {
    return <span>Browser doesn't support speech recognition.</span>;
  }

  return (
    <div>
      <p>Microphone: {listening ? 'on' : 'off'}</p>
      <button onClick={SpeechRecognition.startListening}>Start</button>
      <button onClick={SpeechRecognition.stopListening}>Stop</button>
      <button onClick={resetTranscript}>Reset</button>
      <p>{transcript}</p>
    </div>
  );
};
export default Dictaphone;
```

รูปที่ 3.3 ตัวอย่างโค้ดระบบแปลงเสียงเป็นข้อความ

ตัวอย่างโค้ด (ดังแสดงในรูปที่ 3.3) มีองค์ประกอบและการทำงานของโค้ด ดังนี้

1) useSpeechRecognition เป็น Hook ที่ใช้จัดการแปลงเสียงเป็นข้อความและสถานะของไมโครโฟน
2) ตัวแปร transcript จะเก็บข้อความที่แปลงจากเสียง
3) ตัวแปร listening ใช้ตรวจสอบว่าไมโครโฟนกำลังเปิดหรือปิด
4) ฟังก์ชัน resetTranscript ใช้ล้างข้อความที่แปลงจากเสียง
5) ฟังก์ชัน browserSupportsSpeechRecognition ใช้ตรวจสอบว่าเบราว์เซอร์ของผู้ใช้รองรับการแปลงเสียงหรือไม่ หากไม่รองรับจะแสดงข้อความแจ้งเตือน

3.2 Tailwind CSS

Tailwind CSS [7] คือ CSS Framework ที่มีรูปแบบการทำงานแบบ Utility-First โดย Tailwind CSS ช่วยให้ผู้พัฒนาสามารถสร้าง UI ด้วยตัวเองได้อย่างรวดเร็ว และยังสามารถปรับรายละเอียดปลีกย่อยได้ง่าย เนื่องจากมาพร้อมกับ class สำเร็จรูปทำให้สามารถใช้งานได้ในทันที

ตัวอย่างการใช้งาน (ดังแสดงในรูปที่ 3.4) เป็นคำสั่งที่ใช้ในการสร้างปุ่มที่มีสีสันต่าง ๆ ด้วย Tailwind CSS สามารถทำได้โดยใช้คลาสที่กำหนดสีพื้นหลัง, สีตัวอักษร, รูปแบบฟอนต์, และขนาดของปุ่ม โดยที่ไม่จำเป็นต้องใช้ CSS ที่ซับซ้อน เช่น bg-blue-500 คือการกำหนดสีพื้นหลังของปุ่มให้เป็นเป็นสีฟ้า, text-gray-900 คือการกำหนดสีตัวอักษรให้เป็นสีเทาเข้ม, font-bold คือการทำให้ตัวอักษรมีความหนา, py-2 px-4 คือการกำหนดระยะห่างภายในปุ่มในแนวตั้งและแนวนอน, rounded คือการทำให้มุมของปุ่มมีความมน และ mr-2 คือการกำหนดระยะห่างระหว่างปุ่มแต่ละปุ่ม โดยผลลัพธ์ของโค้ด (ดังแสดงในรูปที่ 3.5) จะทำการแสดงปุ่มที่มีสีและลักษณะตามคำสั่งที่กำหนด (ดังแสดงในรูปที่ 3.4)

```
<div>
      <button className='bg-blue-500 text-gray-900 font-bold py-2 px-4 rounded mr-2'>
        Info
      </button>
      <button className='bg-green-500 text-gray-900 font-bold py-2 px-4 rounded mr-2'>
        Success
      </button>
      <button className='bg-yellow-500 text-gray-900 font-bold py-2 px-4 rounded mr-2'>
        Warning
      </button>
      <button className='bg-red-500 text-gray-900 font-bold py-2 px-4 rounded mr-2'>
        Error
      </button>
    </div>
```

รูปที่ 3.4 ตัวอย่างคำสั่งสร้างปุ่มด้วย Tailwind CSS

รูปที่ 3.5 ตัวอย่างผลลัพธ์การสร้างปุ่มด้วย Tailwind CSS

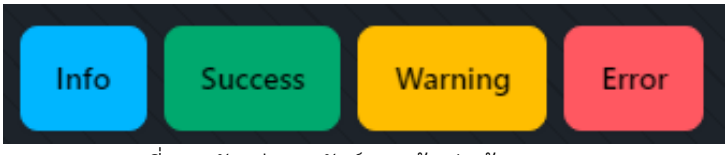
3.3 DaisyUI

เป็นปลั๊กอินของ Tailwind CSS ที่ออกแบบมาเพื่อช่วยสร้าง UI ได้ง่ายขึ้น โดย DaisyUI [8] มาพร้อมกับ component สำเร็จรูป เช่น Avatar, Button, Card, Dropdown เป็นต้น ที่มีการออกแบบตามหลักการของ Tailwind CSS ซึ่งทำให้ง่ายต่อการปรับแต่งและใช้งาน แต่หากต้องการปรับแต่งหรือเพิ่มฟังก์ชันที่มีความเฉพาะเจาะจง อาจต้องเขียน CSS หรือโค้ดเพิ่มเติม เพราะ DaisyUI มีการออกแบบที่ตายตัว และอาจจำกัดความยืดหยุ่นในการปรับเปลี่ยน แก้ไข

ตัวอย่างการใช้งาน (ดังแสดงในรูปที่ 3.6) เป็นคำสั่งที่ใช้ในการสร้างปุ่มที่มีสีสันต่าง ๆ โดยใช้คลาสจาก DaisyUI เช่น btn-info สำหรับปุ่มสีฟ้า, btn-success สำหรับปุ่มสีเขียว, btn-warning สำหรับปุ่มสีเหลือง และ btn-error สำหรับปุ่มสีแดง ซึ่งทั้งหมดนี้ช่วยให้ในการสร้างปุ่มทำได้ง่ายและรวดเร็ว (ดังแสดงในรูปที่ 3.7)

```
<div>
        <button className='btn btn-info'>Info</button>
        <button className='btn btn-success'>Success</button>
        <button className='btn btn-warning'>Warning</button>
        <button className='btn btn-error'>Error</button>
    </div>
```

รูปที่ 3.6 ตัวอย่างคำสั่งสร้างปุ่มด้วย DaisyUI



รูปที่ 3.7 ตัวอย่างผลลัพธ์การสร้างปุ่มด้วย DaisyUI

3.4 Docker

Docker [9] เป็นซอฟต์แวร์ที่ช่วยให้การสร้าง ทดสอบ และติดตั้งแอปพลิเคชันทำได้อย่างรวดเร็ว ด้วยการใช้คอนเทนเนอร์ (Container) ซึ่งจะมีทุกสิ่งที่ซอฟต์แวร์ต้องการเรียกใช้งาน รวมทั้งไลบรารี เครื่องมือสำหรับระบบ โค้ด และรันไทม์ คอนเทนเนอร์จะทำงานในลักษณะที่แยกจากกันและสามารถนำไปใช้งานในสภาพแวดล้อมใดก็ได้ โดยไม่ต้องกังวลเรื่องความแตกต่างของระบบปฏิบัติการหรือเครื่องมือที่ใช้บนเครื่องนั้น ๆ ทำให้แอปพลิเคชันสามารถทำงานได้อย่างมีประสิทธิภาพและเสถียรในทุกสภาพแวดล้อม โดย Docker มีลักษณะของโลโก้ (ดังแสดงในรูปที่ 3.8)

สำหรับหลักการทำงานของ Docker นั้นจะประกอบด้วยส่วนหลัก ๆ ที่สำคัญ ได้แก่

1) Docker Image จะเป็นแม่แบบที่ใช้ในการสร้างคอนเทนเนอร์ โดยจะประกอบไปด้วยโค้ด ไลบรารี และสิ่งที่จำเป็นทั้งหมดในการรันแอปพลิเคชัน
2) Docker Container จะเป็นหน่วยการทำงานที่ทำงานแยกจากกัน คอนเทนเนอร์จะถูกสร้างจาก Docker Image และจะทำงานในสภาพแวดล้อมที่แยกจากกัน
3) Docker Engine เป็นซอฟต์แวร์ที่ใช้ในการสร้างและจัดการ Docker Container โดยที่ Docker Engine จะทำงานร่วมกับระบบปฏิบัติการ

รูปที่ 3.8 Docker

สำหรับขั้นตอนในการติดตั้งและใช้งาน Docker สามารถทำได้ดังนี้

1) ทำการโหลด Docker จากเว็บไซต์ทางการ และเลือกเวอร์ชันที่เหมาะสมกับระบบปฏิบัติการที่ผู้ใช้ใช้งานอยู่
2) ติดตั้ง Docker โดยทำตามขั้นตอนที่แสดงอยู่ในตัวติดตั้ง
3) เปิดแอปพลิเคชัน Docker และตรวจสอบให้แน่ใจว่า Docker กำลังทำงานอยู่
4) ตรวจสอบการติดตั้งด้วยการใช้คำสั่งบนเทอร์มินัลหรือ Command Prompt (ดังแสดงในรูปที่ 3.9) หากติดตั้งสำเร็จจะมีการแสดงเวอร์ชันของ Docker ที่กำลังใช้งาน

```
C:\Users\USER>docker --version
Docker version 27.3.1, build ce12230
```

รูปที่ 3.9 ตัวอย่างการเช็คเวอร์ชันของ Docker

5) ทดสอบการใช้งาน Docker ด้วยการรันคำสั่ง (ดังแสดงในรูปที่ 3.10) โดยในคำสั่งนี้จะเป็นการดึง Container ทดสอบจาก Docker Hub และทำการรันเพื่อทำการยืนยันว่า Docker ทำงานได้ถูกต้อง

```
C:\Users\USER>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:305243c734571da2d100c8c8b3c3167a098cab6049c9a5b066b6021a60fcb966
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

รูปที่ 3.10 ตัวอย่างทดสอบการใช้งาน Docker

3.5 PyThaiNLP

PyThaiNLP [10] เป็นไลบรารี Python ที่ถูกพัฒนาขึ้นมาเพื่อการประมวลผลภาษาธรรมชาติของภาษาไทย โดยไลบรารีนี้มีฟังก์ชันหลากหลายที่ช่วยในการจัดการข้อมูลภาษาไทยได้อย่างมีประสิทธิภาพ ตัวอย่างฟังก์ชันที่สำคัญ ได้แก่ การตรวจสอบตัวสะกด การแท็กส่วนของคำพูด (Part-of-Speech Tagging) การตัดคำ (Word Segmentation) การแปลงคำอ่าน (Transliteration) การแยกพยางค์ และการสร้างพจนานุกรมเฉพาะ ซึ่งฟังก์ชันเหล่านี้ทำให้นักพัฒนาสามารถจัดการและวิเคราะห์ข้อมูลทางภาษาศาสตร์ได้ง่ายขึ้น โดย PyThaiNLP มีลักษณะของโลโก้ (ดังแสดงในรูปที่ 3.11)

รูปที่ 3.11 PyThaiNLP

ตัวอย่างขั้นตอนการเรียกใช้งานฟังก์ชัน Syllable_Tokenize จากไลบรารี่ PyThaiNLP ทำได้โดยการนำเข้าโมดูล Syllable_Tokenize เข้ามาใช้งาน โดยการใช้ Engine ชื่อว่า newmm ที่เป็น Engine เริ่มต้น ในการตัดพยางค์ในข้อความ (ดังแสดงในรูปที่ 3.12) และแสดงผลลัพธ์หลังจากการตัดพยางค์ในข้อความ (ดังแสดงในรูปที่ 3.13)

```
from pythainlp.tokenize import syllable_tokenize
text = "การเรียนรู้ไม่มีที่สิ้นสุดถ้าเรามุ่งมั่น"
words = syllable_tokenize(text)
print(words)
```

รูปที่ 3.12 ตัวอย่างการตัดพยางค์โดยการใช้ Syllable_Tokenize

```
['การ', 'เรียน', 'รู้', 'ไม่', 'มี', 'ที่', 'สิ้น', 'สุด', 'ถ้า', 'เรา', 'มุ่ง', 'มั่น']
```

รูปที่ 3.13 ตัวอย่างผลลัพธ์การตัดพยางค์โดยการใช้ Syllable_Tokenize

ตัวอย่างการเรียกใช้งานฟังก์ชัน KhaveeVerifier จากไลบรารี่ PyThaiNLP โดยการนำเข้าโมดูล KhaveeVerifier เข้ามาใช้งานในการตรวจสอบคำคล้องจอง (ดังแสดงในรูปที่ 3.14) และแสดงผลลัพธ์หลังจากการตรวจสอบคำคล้องจอง (ดังแสดงในรูปที่ 3.15)

```python
from pythainlp.khavee import KhaveeVerifier
kv = KhaveeVerifier()
word1 = "กล่อง"
word2 = "กล้อง"
result = kv.is_sumpus(word1, word2)
print(result)
```

รูปที่ 3.14 ตัวอย่างการตรวจสอบคำคล้องจองโดยการใช้ KhaveeVerifier

```
True
```

รูปที่ 3.15 ตัวอย่างผลลัพธ์การตรวจสอบคำคล้องจองโดยการใช้ KhaveeVerifier

3.6 รูปสระในภาษาไทย

รูปสระ [11] หมายถึง เครื่องหมายที่ใช้เขียนแทนเสียงสระ โดยรูปสระในภาษาไทยจะมี 21 รูป (ดังแสดงในตารางที่ 3.1) ดังนี้

ตารางที่ 3.1 สระในภาษาไทย

รูปสระในภาษาไทย 21 รูป			
1	ะ เรียกว่า วิสรรชนีย์	12	ใ เรียกว่า ไม้ม้วน
2	◌ั เรียกว่า ไม้หันอากาศ หรือ ไม้ผัด	13	ไ เรียกว่า ไม้มลาย
3	◌็ เรียกว่า ไม้ไต่คู้	14	โ เรียกว่า ไม้โอ
4	า เรียกว่า ลากข้าง	15	อ เรียกว่า ตัวออ
5	◌ิ เรียกว่า พินทุอิ	16	ย เรียกว่า ตัวยอ
6	◌่ เรียกว่า ฝนทอง	17	ว เรียกว่า ตัววอ
7	◌่◌่ เรียกว่า ฟันหนู	18	ฤ เรียกว่า ตัว ฤ (รึ)
8	◌ํ เรียกว่า นฤคหิต หรือ หยาดน้ำค้าง	19	ฤๅ เรียกว่า ตัว ฤๅ (รือ)
9	◌ุ เรียกว่า ตีนเหยียด	20	ฦ เรียกว่า ตัว ฦ (ลึ) ปัจจุบันเลิกใช้แล้ว
10	◌ู เรียกว่า ตีนคู้	21	ฦๅ เรียกว่า ตัว ฦๅ (ลือ) ปัจจุบันเลิกใช้แล้ว
11	เ เรียกว่า ไม้หน้า		

3.7 มาตราตัวสะกด

มาตราตัวสะกด [12] หมายถึง พยัญชนะที่ประกอบอยู่ท้ายสระและมีเสียงประสมเข้ากับสระ ในมาตราตัวสะกดภาษาไทยมีอยู่ 8 มาตรา (ดังแสดงในตารางที่ 3.2) ดังนี้

ตารางที่ 3.2 มาตราตัวสะกด

มาตราตัวสะกด 8 มาตรา			
1	แม่ กก ได้แก่ ก ข ค ฆ	5	แม่ กง ได้แก่ ง
2	แม่ กด ได้แก่ ด จ ช ซ ฎ ฏ ฐ ฑ ฒ ต ถ ท ธ ศ ษ ส	6	แม่ กม ได้แก่ ม
3	แม่ กน ได้แก่ น ญ ณ ร ล ฬ	7	แม่ เกย ได้แก่ ย
4	แม่ กบ ได้แก่ บ ป พ ฟ ภ	8	แม่ เกอว ได้แก่ ว

3.8 คำคล้องจอง

คำคล้องจอง [13] คือ คำที่มีเสียงสระเดียวกัน และถ้ามีตัวสะกดจะต้องอยู่ในมาตราเดียวกัน มีเสียงวรรณยุกต์ต่างกันได้ (คำคล้องจองในบทประพันธ์ร้อยกรอง เรียกว่า สัมผัส) สามารถแบ่งลักษณะของคำคล้องจองได้ ดังนี้

3.8.1 คำคล้องจอง 1 พยางค์

คือ คำที่มีเสียงสระและเสียงพยัญชนะท้ายเหมือนกันเช่น ตา ยา มา ปลา

3.8.2 คำคล้องจอง 2 พยางค์

คือ กลุ่มคำที่มีอยู่ 2 พยางค์ โดยที่พยางค์สุดท้ายของกลุ่มคำแรก มีเสียงที่คล้องจองกับพยางค์แรกของกลุ่มคำต่อไป เช่น (ยิ้มแย้ม แจ่มใส่), (มานี มีนา), (พรุ่งนี้ มีงาน)

3.8.3 คำคล้องจอง 3 พยางค์

คือ กลุ่มคำที่มีอยู่ 3 พยางค์ โดยที่พยางค์สุดท้ายของกลุ่มคำแรก มีเสียงที่คล้องจองกับพยางค์แรกของกลุ่มคำต่อไป เช่น (อ่านหนังสือ คือปัญญา), (ตั้งใจเรียน เพียรศึกษา), (ทุกนาที มีคุณค่า)

3.8.4 คำคล้องจอง 4 พยางค์

คือ กลุ่มคำที่มีอยู่ 4 พยางค์ โดยที่พยางค์สุดท้ายของกลุ่มคำแรก มีเสียงที่คล้องจองกับพยางค์แรกหรือพยางค์สองของกลุ่มคำต่อไป เช่น (กินอาหารดี มีอายุยืน), (เจ้าปลาตัวน้อย ล่องลอยในน้ำ), (ไร้โทษเพิ่มสุข หมดทุกข์แข็งแรง)

กฎการใช้คำคล้องจองในภาษาไทยไม่จำเป็นต้องเคร่งครัดโดยเฉพาะพยางค์สุดท้ายที่เป็นจุดสำคัญในการสร้างความคล้องจอง อย่างไรก็ตามในการเขียนกลอนหรือวรรณกรรมบางประเภทจะมีการบังคับให้ปฏิบัติตามกฎที่ชัดเจน ผู้เขียนสามารถใช้ความคิดสร้างสรรค์ในการนำเสนอคำคล้องจองได้ตามต้องการโดยไม่ต้องยึดติดกับกฎที่ตั้งไว้ หากมีการศึกษาหรือสอนเกี่ยวกับคำคล้องจอง ควรเน้นให้ความสำคัญกับความไพเราะ และลื่นไหลของการสื่อสาร

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

ในบทที่ 4 นี้จะกล่าวถึงขั้นตอนการดำเนินงานต่าง ๆ ของโครงงานวิจัยโดยแสดงในรูปแบบของตารางเวลาตั้งแต่ขั้นตอนเริ่มต้นจนถึงขั้นตอนสุดท้าย

4.1 ขั้นตอนการดำเนินงาน

4.1.1 รวบรวมข้อมูลและกำหนดขอบเขตของงาน

- ศึกษาค้นคว้าหาข้อมูลที่เกี่ยวข้อง และกำหนดขอบเขตของงาน

4.1.2 ศึกษางานและทฤษฎีที่เกี่ยวข้อง

- เรียนรู้การใช้งาน React.js
- ศึกษาการใช้งาน PyThaiNLP
- ศึกษาคำคล้องจอง

4.1.3 ออกแบบหน้าเว็บแอปพลิเคชันและระบบงาน

- ออกแบบโครงสร้างของเว็บแอพลิเคชัน
- ออกแบบหน้าเว็บสำหรับผู้ใช้งาน

4.1.4 พัฒนาโปรแกรม

- สร้างเว็บแอปพลิเคชันคำคล้องจอง

4.1.5 ทดสอบและแก้ไขข้อผิดพลาด

- ทดสอบโปรแกรม และแก้ไขข้อผิดพลาดของโปรแกรม

4.1.6 จัดทำเอกสารประกอบโครงงาน

- จัดทำเอกสารประกอบโครงงานตั้งแต่เริ่มต้นจนถึงขั้นตอนสุดท้าย

4.2 แผนการดำเนินงาน

ในโครงงานปริญญานิพนธ์นี้ได้มีแผนการดำเนินงานตั้งแต่เดือนเมษายน พ.ศ. 2567 ไปจนถึงเดือนพฤศจิกายน พ.ศ. 2567 (ดังแสดงในตารางที่ 4.1)

ตารางที่ 4.1 แผนการดำเนินงาน

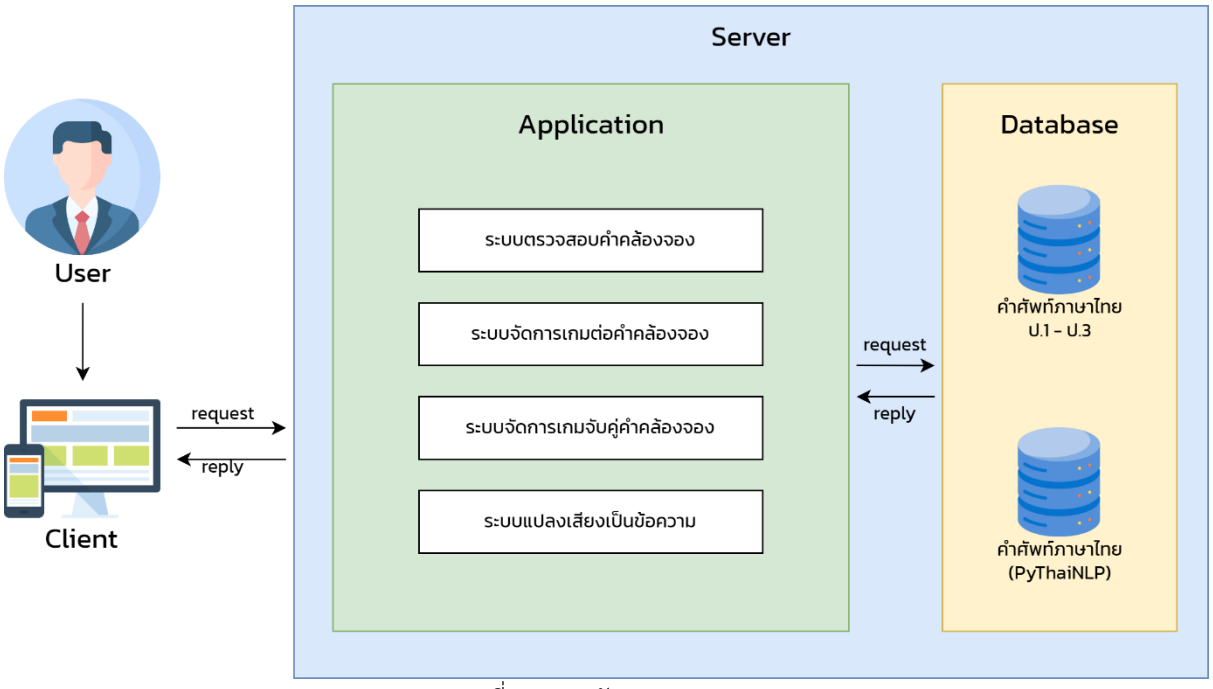
ขั้นตอนการดำเนินงาน	เดือน							
	เม.ย. 2567	พ.ค. 2567	มิ.ย. 2567	ก.ค. 2567	ส.ค. 2567	ก.ย. 2567	ต.ค. 2567	พ.ย. 2567
รวบรวมข้อมูลและกำหนดขอบเขตของงาน	↔	↔						
ศึกษางานและทฤษฎีที่เกี่ยวข้อง	↔	↔	↔					
ออกแบบหน้าเว็บแอปพลิเคชันและระบบงาน	↔	↔	↔					
พัฒนาโปรแกรม			↔	↔	↔	↔	↔	↔
ทดสอบและแก้ไขข้อผิดพลาด			↔	↔	↔	↔	↔	↔
จัดทำเอกสารประกอบโครงงาน	↔	↔	↔	↔	↔	↔	↔	↔

บทที่ 5 การวิเคราะห์และออกแบบระบบ

ในบทที่ 5 นี้จะเป็นการอธิบายหลักการทำงานของระบบและโครงสร้างการทำงานของเว็บแอปพลิเคชันตรวจคำคล้องจอง โดยจะมีรายละเอียดดังต่อไปนี้

5.1 สถาปัตยกรรมของระบบ

โครงงานนี้มีโครงสร้างการทำงานของระบบ ดังนี้

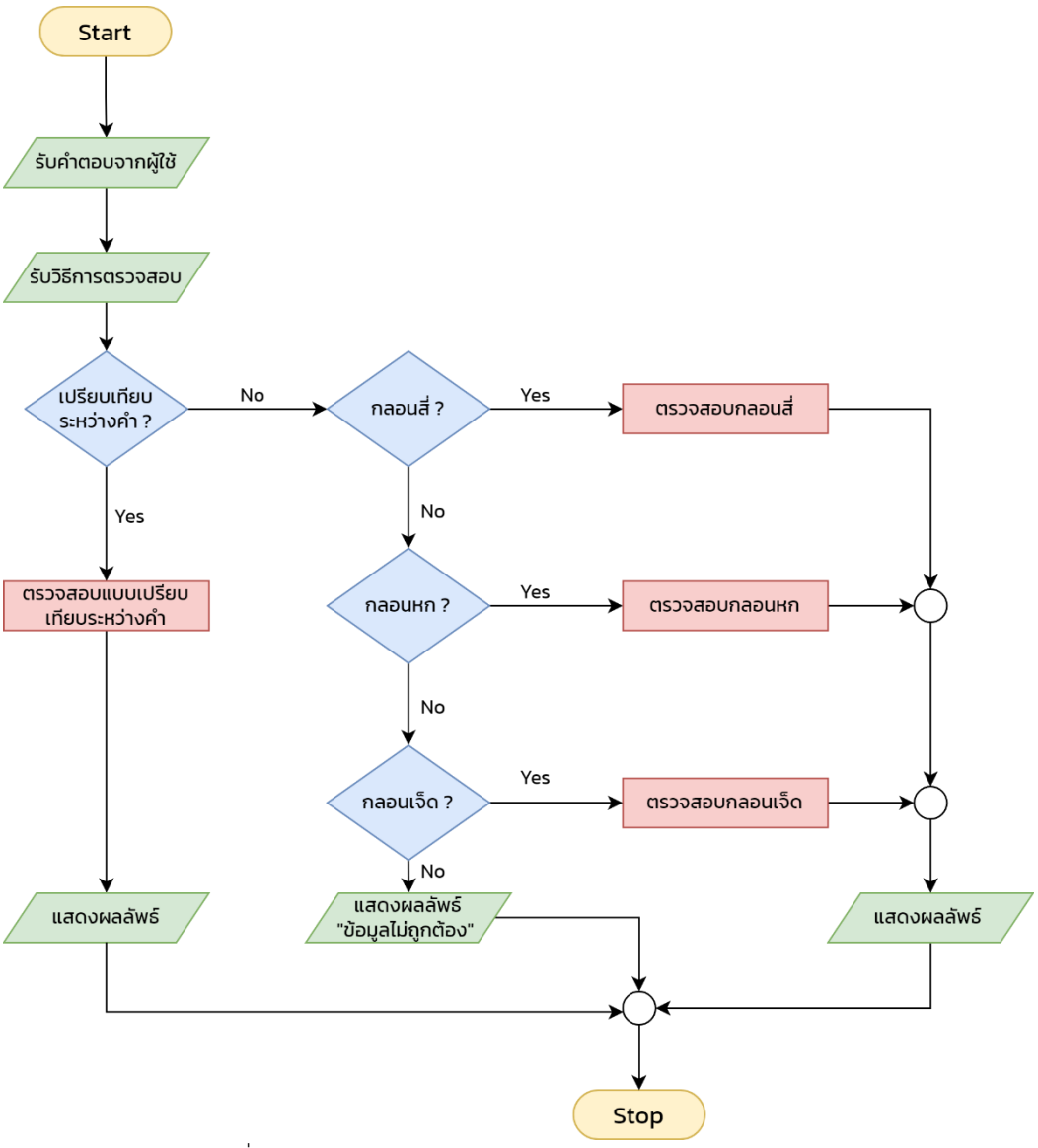


รูปที่ 5.1 สถาปัตยกรรมระบบ

สถาปัตยกรรมระบบตรวจสอบคำคล้องจอง (ดังแสดงในรูปที่ 5.1) สามารถแบ่งการทำงานออกเป็นส่วนต่าง ๆ ที่เชื่อมต่อระหว่าง Client และ Server ได้ดังนี้

1) ฝั่งผู้ใช้งาน (User) : ผู้ใช้สามารถเข้าถึงระบบผ่าน อุปกรณ์ Client เช่น คอมพิวเตอร์หรือโทรศัพท์มือถือ โดยใช้เว็บเบราว์เซอร์ในการส่งคำขอ (Request) ไปยังเซิร์ฟเวอร์ และรับผลลัพธ์ (Reply) กลับมาแสดงผล
2) ฝั่งเซิร์ฟเวอร์ (Server) : เซิร์ฟเวอร์แบ่งออกเป็นสองส่วนหลัก คือ Application Layer และ Database Layer
3) แอปพลิเคชัน (Application) : ส่วนนี้ทำหน้าที่ประมวลผลคำขอจากผู้ใช้และจัดการการทำงานต่าง ๆ ของระบบ โดยประกอบไปด้วย 4 ระบบย่อย ได้แก่ ระบบตรวจสอบคำคล้องจอง ระบบจัดการเกมต่อคำคล้องจอง ระบบจัดการเกมจับคู่คำคล้องจอง และระบบแปลงเสียงเป็นข้อความ
4) ฐานข้อมูล (Database) : เป็นส่วนที่ใช้ในการจัดเก็บคำศัพท์สำหรับการตรวจสอบคำคล้องจองและเกม โดยแบ่งออกเป็น 2 ฐานข้อมูล ได้แก่ ฐานข้อมูลคำศัพท์ภาษาไทยระดับประถมศึกษา (ป.1 – ป.3) และฐานข้อมูลคำศัพท์ภาษาไทยจาก PyThaiNLP

5.2 การทำงานของระบบตรวจสอบคำคล้องจอง



รูปที่ 5.2 Flowchart การทำงานของระบบตรวจสอบคำคล้องจอง

ขั้นตอนการทำงานของระบบตรวจสอบคำคล้องจอง (ดังแสดงในรูปที่ 5.2) มีขั้นตอนดังนี้

1) ระบบเริ่มต้นโดยรอรับข้อมูลจากผู้ใช้งาน
2) ผู้ใช้งานทำการป้อนคำตอบที่ต้องการตรวจสอบเข้ามาในระบบ โดยรูปแบบของข้อมูลที่รับมีดังนี้
   - กรณีการตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ ระบบจะทำการรับข้อความที่เป็นคำ 2 คำที่ต้องการตรวจสอบ เช่น “การงาน บ้านเมือง”
   - กรณีการตรวจสอบกลอน ระบบจะทำการรับข้อความที่เป็นบทกลอน เช่น

     ดวงจันทร์วันเพ็ญ ลอยเด่นบนฟ้า
     แสงนวลเย็นตา พาใจหฤหรรษ์
     ชักชวนเพื่อนยา มาเล่นร่วมกัน
     เด็กน้อยสุขสันต์ บันเทิงเริงใจ

3) ผู้ใช้งานเลือกวิธีการตรวจสอบคำคล้องจองที่ต้องการ ได้แก่
   - การตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ
   - การตรวจสอบกลอนสี่
   - การตรวจสอบกลอนหก
   - การตรวจสอบกลอนเจ็ด
4) ระบบจะทำการตรวจสอบคำหรือบทกลอนตามวิธีที่ผู้ใช้งานเลือก
   - หากผู้ใช้เลือกวิธีการตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ ระบบจะทำการตรวจสอบว่าคำสองคำที่ได้รับมาคล้องจองกันหรือไม่
   - หากผู้ใช้เลือกวิธีการตรวจสอบกลอน ระบบจะทำการตรวจสอบโครงสร้างและการคล้องจองในแต่ละวรรคและบทของกลอน
5) ระบบแสดงผลลัพธ์หลังจากการตรวจสอบ
6) ระบบจบการทำงานเมื่อแสดงผลลัพธ์เรียบร้อยแล้ว

5.2.1 การทำงานของระบบตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ

ในระบบตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ จะมีขั้นตอนการทำงานของโค้ดดังต่อไปนี้

```
@app.get("/Check-rhyming-words")
def check_rhyme_words(word1: str, word2: str):
    try:
        result = Check_Rhyme_Word(word1, word2)
        return {"result": result}
    except Exception as e:
        return {"error": str(e)}
```

รูปที่ 5.3 โค้ดตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ (1)

ฟังก์ชัน check_rhyme_words (ดังแสดงในรูปที่ 5.3) ทำหน้าที่ในการรับคำสองคำจากผู้ใช้งานเพื่อตรวจสอบความคล้องจองระหว่างคำทั้งสอง โดยคำทั้งสองจะถูกส่งมาผ่านพารามิเตอร์ word1 และ word2 หลังจากที่ฟังก์ชันได้รับค่าพารามิเตอร์ทั้งสองแล้ว ฟังก์ชันจะทำการเรียกใช้ฟังก์ชัน Check_Rhyme_Word ซึ่งมีหน้าที่ตรวจสอบความคล้องจองระหว่างคำทั้งสอง หากคำทั้งสองมีความคล้องจองกันฟังก์ชันจะคืนค่า True กลับมา แต่หากคำทั้งสองไม่คล้องจองกัน ฟังก์ชันจะคืนค่า False พร้อมทั้งส่งผลลัพธ์กลับไปยังผู้เรียกใช้ในรูปแบบ JSON เพื่อแสดงข้อมูลผลการตรวจสอบหรือข้อผิดพลาดที่อาจเกิดขึ้นระหว่างกระบวนการตรวจสอบคำ

```python
from pythainlp.tokenize import syllable_tokenize
from pythainlp.khavee import KhaveeVerifier

def Check_Rhyme_Word(word1: str, word2: str):
    # ตัดพยางค์ของแต่ละคำ
    tokens1 = syllable_tokenize(word1)
    tokens2 = syllable_tokenize(word2)

    kv = KhaveeVerifier()
    result = False
    length = len(tokens1)

    # กรณีคำที่สั้นกว่าหรือเท่ากับ 4 พยางค์
    if length <= 4:
        if length == 1:
            result = kv.is_sumpus(tokens1[0], tokens2[0])
        elif length == 2:
            result = kv.is_sumpus(tokens1[-1], tokens2[0])
        elif length == 3:
            result = kv.is_sumpus(tokens1[-1], tokens2[0])
        elif length == 4:
            result = any(kv.is_sumpus(tokens1[-1], tokens2[index]) for index in [0,1])
    else:
        # เช็คพยางค์สุดท้ายของ tokens1 กับพยางค์ใน tokens2 ที่กำหนดโดย length
        for index in range(length):
            if kv.is_sumpus(tokens1[-1], tokens2[index]):
                result = True

    return result
```

รูปที่ 5.4 โค้ดตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ (2)

ฟังก์ชัน Check_Rhyme_word ทำการรับคำสองคำที่ต้องการตรวจสอบว่าคล้องจองกันหรือไม่ โดยคำทั้งสองคำนี้จะถูกเก็บไว้ในตัวแปร word1 และ word2 หลังจากรับคำทั้งสองคำแล้ว ฟังก์ชันจะทำการแบ่งคำแต่ละคำออกเป็นพยางค์ ๆ โดยใช้เครื่องมือที่ชื่อว่า syllable_tokenize ซึ่งเป็นส่วนหนึ่งของไลบรารี PyThaiNLP ที่ใช้สำหรับประมวลผลภาษาไทย พยางค์ที่ได้จากการแบ่งจะถูกเก็บไว้ในตัวแปร tokens1 และ tokens2 เมื่อได้พยางค์ของทั้งสองคำแล้ว จะทำการตรวจสอบว่าคำทั้งสองคำนั้นคล้องจองกันหรือไม่ (ดังแสดงในรูปที่ 5.4) หากคล้องจองฟังก์ชันจะคืนค่า True กลับมา ซึ่งหมายความว่าคำทั้งสองคำคล้องจองกัน แต่ถ้าไม่คล้องจองกันฟังก์ชันจะคืนค่า False กลับมา

5.2.2 การทำงานของระบบตรวจสอบกลอน

ในระบบตรวจสอบคำกลอน จะมีขั้นตอนการทำงานของโค้ดดังต่อไปนี้

```
@app.post("/check-klon")
async def check_klon(input_klon: KlonInput, k_type: int):
    try:
        if k_type == 4:
            return check_klon4(input_klon.klon)
        elif k_type == 6:
            return check_klon6(input_klon.klon)
        elif k_type == 7:
            return check_klon7(input_klon.klon)
        else:
            return {"error": "ข้อมูลไม่ถูกต้อง"}
    except Exception as e:
        return {"error": str(e)}
```

รูปที่ 5.5 โค้ดตรวจสอบกลอน

ฟังก์ชัน check_klon (ดังแสดงในรูปที่ 5.5) ทำหน้าที่รับข้อมูลบทกลอนและวิธีการตรวจสอบจากผู้ใช้งาน โดยข้อมูลดังกล่าวจะถูกส่งมาในรูปของพารามิเตอร์ input_klon ซึ่งเป็นข้อมูลบทกลอน และ k_type ซึ่งระบุประเภทของกลอนหรือวิธีการตรวจสอบ เมื่อฟังก์ชันได้รับพารามิเตอร์ทั้งสองเรียบร้อยแล้ว จะดำเนินการตรวจสอบบทกลอนตามประเภทที่ผู้ใช้งานกำหนด โดยเรียกใช้ฟังก์ชันตรวจสอบเฉพาะสำหรับแต่ละประเภท เช่น กลอนสี่ กลอนหก หรือกลอนเจ็ด หลังจากกระบวนการตรวจสอบเสร็จสิ้น ฟังก์ชันจะส่งผลลัพธ์การตรวจสอบกลับไปยังผู้เรียกใช้ในรูปแบบ JSON เพื่อแสดงผลข้อมูลผลการตรวจสอบหรือหรือข้อผิดพลาดที่อาจเกิดขึ้นระหว่างกระบวนการตรวจสอบคำ

การตรวจสอบกลอนสี่

ฟังก์ชัน check_klon4 ทำหน้าที่ในการตรวจสอบกลอนสี่ (กลอนสุภาพที่มี 4 วรรค) โดยแบ่งการทำงานออกเป็นส่วนต่าง ๆ เพื่อวิเคราะห์บทกลอนและตรวจสอบความถูกต้องตามกฎสัมผัส

ฟังก์ชันนี้จะรับข้อมูลกลอนสี่จากผู้ใช้ในรูปแบบข้อความ (string) แล้วแยกออกเป็นบท ๆ เพื่อตรวจสอบแต่ละบทอย่างละเอียด รวมถึงตรวจสอบข้อผิดพลาดต่าง ๆ ที่อาจเกิดขึ้น เช่น จำนวนวรรคไม่ครบ หรือสัมผัสคล้องจองไม่ถูกต้องตามกฎ

การทำงานของฟังก์ชัน check_klon4 มีขั้นตอนการทำงานดังนี้

1) ระบบจะรับข้อมูลกลอนสี่ในรูปแบบข้อความ (input_klon) ผู้ใช้งานสามารถป้อนบทกลอนที่ต้องการตรวจสอบได้หลายบท
2) ระบบทำการแยกบทกลอนตามบรรทัดว่าง (\n\n) เพื่อนำไปตรวจสอบเป็นบท ๆ แต่ละบทจะถูกเก็บในรูปแบบ List ของข้อความ (ดังแสดงในรูปที่ 5.6)

```
klon_list = input_klon.split('\n\n')
```

รูปที่ 5.6 โค้ดแยกบทกลอน

3) ในแต่ละบทระบบจะทำการแยกวรรคตามบรรทัด (\n) และใช้ฟังก์ชัน syllable_tokenize() ในการแยกพยางค์ของแต่ละวรรคเก็บไว้ในลิสต์ syllables (ดังแสดงในรูปที่ 5.7)

```
klon_lines = klon.split('\n')
syllables = [syllable_tokenize(line) for line in klon_lines]
```

รูปที่ 5.7 โค้ดแยกวรรคและพยางค์

4) ตรวจสอบจำนวนวรรคในแต่ละบทหากบทนั้นมีวรรคไม่ครบ 4 วรรค จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.8)

```
if len(syllables) != 4:
            result["ข้อผิดพลาด"].append(f"จำนวนวรรคของบท {index+1} ไม่ครบตามที่ต้องการ โปรดตรวจสอบและแก้ไข")
```

รูปที่ 5.8 โค้ดตรวจสอบจำนวนวรรคในแต่ละบท

5) ตรวจสอบจำนวนพยางค์ในแต่ละวรรคแต่ละวรรคต้องมีจำนวนพยางค์ครบ 4 พยางค์ หากขาดหรือเกิน จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.9)

```
for i in range(4):
    if len(syllables[i]) != 4:
        result["ข้อผิดพลาด"].append(
            f"วรรคที่ {i+1} ของบท {index+1} จำนวนพยางค์ขาดหรือเกินที่ต้องการ (4 พยางค์) ผลลัพธ์การตรวจสอบอาจไม่ถูกต้อง"
        )
```

รูปที่ 5.9 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนสี่

6) ตรวจสอบสัมผัสตามกฎกลอนสี่ ฟังก์ชันจะตรวจสอบสัมผัสในแต่ละวรรคตามกฎสัมผัสของกลอนสี่ โดยใช้ฟังก์ชัน kv.is_sumpus เพื่อตรวจสอบการสัมผัส โดยการตรวจกลอนสี่มีขั้นตอนการตรวจสอบดังนี้

กฎที่ 1: คำสุดท้ายของวรรคที่ 1 สัมผัสกับคำที่ 1 หรือ 2 ของวรรคที่ 2 (ดังแสดงในรูปที่ 5.10)

กฎที่ 2: คำสุดท้ายของวรรคที่ 2 สัมผัสกับคำสุดท้ายของวรรคที่ 3 (ดังแสดงในรูปที่ 5.11)

กฎที่ 3: คำสุดท้ายของวรรคที่ 3 สัมผัสกับคำที่ 1 หรือ 2 ของวรรคที่ 4 (ดังแสดงในรูปที่ 5.12)

กฎที่ 4: คำสุดท้ายของวรรคที่ 4 ของบทปัจจุบันสัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบทถัดไป (ดังแสดงในรูปที่ 5.13)

```python
#ตรวจสอบกฎที่ 1
if len(syllables[0]) < 3 or len(syllables[1]) < 3:
    result['ข้อผิดพลาด'].append(
        "กฎที่ 1 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 1 สัมผัสคล้องจองกับคำที่ 1 หรือ 2 ของวรรคที่ 2)"
    )
else:        (variable) syllables: list[List[str]]
    if len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][0])) and (kv.is_sumpus(syllables[0][-1], syllables[1][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 1"
            f"('{syllables[1][0]}') หรือ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
    elif len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][0])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 1 ('{syllables[1][0]}') ของวรรคที่ 2"
        )
    elif len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ❌: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') ไม่สัมผัสกับคำที่ 1 ('{syllables[1][0]}') หรือ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
```

รูปที่ 5.10 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 1

```python
if len(syllables[1]) < 3 or len(syllables[2]) < 3:
    result['ข้อผิดพลาด'].append(
        "กฎที่ 2 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 2 สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3)"
    )
else:
    if len(syllables) >= 3 and kv.is_sumpus(syllables[1][-1], syllables[2][-1]):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ✅: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ❌: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
```

รูปที่ 5.11 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 2

```python
if len(syllables[2]) < 3 or len(syllables[3]) < 3:
    result['ข้อผิดพลาด'].append(
        "กฎที่ 3 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 3 สัมผัสคล้องจองกับคำที่ 1 หรือ 2 ของวรรคที่ 4)"
    )
else:
    if len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][0])) and (kv.is_sumpus(syllables[2][-1], syllables[3][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 1"
            f"('{syllables[3][0]}') หรือ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
    elif len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][0])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 1 ('{syllables[3][0]}') ของวรรคที่ 4"
        )
    elif len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ❌: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') ไม่สัมผัสกับคำที่ 1 ('{syllables[3][0]}') หรือ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
```

รูปที่ 5.12 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 3

```python
#ตรวจสอบกฎที่ 4 คำสุดท้ายของวรรคที่ 4 ของบทปัจจุบันกับคำสุดท้ายของวรรคที่ 2 ของบทถัดไป
if index < len(klon_list) - 1:  # ตรวจสอบว่าบทถัดไปมีอยู่
    next_klon_lines = klon_list[index + 1].split('\n')
    next_syllables = [syllable_tokenize(line) for line in next_klon_lines]
    if len(syllables[3]) < 3 or len(next_syllables[1]) < 3:
        result['ข้อผิดพลาด'].append(
            f"กฎที่ 4 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 4 ของบท {index + 1}"
            f"สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2}"
        )
    else:
        if len(syllables) >= 4 and len(next_syllables) >= 2:
            if kv.is_sumpus(syllables[3][-1], next_syllables[1][-1]):
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ✅: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
            else:
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ❌: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
```

รูปที่ 5.13 โค้ดตรวจสอบกลอนสี่ กฎข้อที่ 4

เมื่อทำการตรวจสอบเสร็จสิ้นผลลัพธ์ของฟังก์ชันจะถูกส่งกลับมาในรูปแบบของ List of Dictionary โดยแต่ละ Dictionary จะประกอบด้วย

- "บท": หมายเลขบทที่ตรวจสอบ
- "ผลการตรวจสอบ": รายละเอียดการตรวจสอบ
- "ข้อผิดพลาด": รายละเอียดข้อผิดพลาดที่พบ

การตรวจสอบกลอนหก

ฟังก์ชัน check_klon6 ทำหน้าที่ในการตรวจสอบกลอนหก โดยแบ่งการทำงานออกเป็นส่วนต่าง ๆ เพื่อวิเคราะห์บทกลอนและตรวจสอบความถูกต้องตามกฎสัมผัส

ฟังก์ชันนี้จะรับข้อมูลกลอนหกจากผู้ใช้ในรูปแบบข้อความ (string) แล้วแยกออกเป็นบท ๆ เพื่อตรวจสอบแต่ละบทอย่างละเอียด รวมถึงตรวจสอบข้อผิดพลาดต่าง ๆ ที่อาจเกิดขึ้น เช่น จำนวนวรรคไม่ครบ หรือสัมผัสคล้องจองไม่ถูกต้องตามกฎ

การทำงานของฟังก์ชัน check_klon6 มีขั้นตอนการทำงานดังนี้

1) ระบบจะรับข้อมูลกลอนหกในรูปแบบของข้อความ (input_klon) ซึ่งผู้ใช้งานสามารถป้อนบทกลอนที่ต้องการตรวจสอบได้หลายบท
2) ระบบทำการแยกบทกลอนตามบรรทัดว่าง (\n\n) เพื่อนำไปตรวจสอบเป็นบท ๆ แต่ละบทจะถูกเก็บในรูปแบบ List ของข้อความ (ดังแสดงในรูปที่ 5.6)
3) ระบบทำการแยกวรรคตามบรรทัด (\n) และใช้ฟังก์ชัน syllable_tokenize() ในการแยกพยางค์ของแต่ละวรรคเก็บไว้ในลิสต์ syllables (ดังแสดงในรูปที่ 5.7)
4) ระบบทำการตรวจสอบจำนวนวรรคในแต่ละบท หากบทนั้นมีวรรคไม่ครบ 4 วรรค จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.8)
5) ระบบทำการตรวจสอบจำนวนพยางค์ในแต่ละวรรคแต่ละวรรคจะต้องมีจำนวนพยางค์ครบ 6 พยางค์ หากขาดหรือเกิน จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.14)

```
for i in range(4):
    if len(syllables[i]) != 6:
        result["ข้อผิดพลาด"].append(
            f"วรรคที่ {i+1} ของบท {index+1} จำนวนพยางค์ขาดหรือเกินที่ต้องการ (6 พยางค์) ผลลัพธ์การตรวจสอบอาจไม่ถูกต้อง"
        )
```

รูปที่ 5.14 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนหก

6) ตรวจสอบสัมผัสตามกฎกลอนหก: ฟังก์ชันจะตรวจสอบสัมผัสในแต่ละวรรคตามกฎสัมผัสของกลอนหก โดยใช้ฟังก์ชัน kv.is_sumpus เพื่อตรวจสอบการสัมผัส โดยการตรวจกลอนหกมีขั้นตอนการตรวจสอบดังนี้

กฎที่ 1: คำสุดท้ายของวรรคที่ 1 สัมผัสกับคำที่ 2 หรือ 3 หรือ 4 ของวรรคที่ 2 (ดังแสดงในรูปที่ 5.15)

กฎที่ 2: คำสุดท้ายของวรรคที่ 2 สัมผัสกับคำสุดท้ายของวรรคที่ 3 (ดังแสดงในรูปที่ 5.16)

กฎที่ 3: คำสุดท้ายของวรรคที่ 3 สัมผัสกับคำที่ 2 หรือ 3 หรือ 4 ของวรรคที่ 4 (ดังแสดงในรูปที่ 5.17)

กฎที่ 4: คำสุดท้ายของวรรคที่ 4 ของบทปัจจุบันสัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบทถัดไป (ดังแสดงในรูปที่ 5.18)

```
if len(syllables[0]) < 4 or len(syllables[1]) < 4:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 1 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 1 สัมผัสคล้องจองกับคำที่ 2 หรือ 3 หรือ 4 ของวรรคที่ 2)"
    )
else:
    if len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][1])) and \
        (kv.is_sumpus(syllables[0][-1], syllables[1][2])) and \
            (kv.is_sumpus(syllables[0][-1], syllables[1][3])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') ไม่สัมผัสกับคำที่ 2"
            f"('{syllables[1][1]}') หรือ 3 ('{syllables[1][2]}') หรือ 4 ('{syllables[1][3]}') ของวรรคที่ 2"
        )
    elif len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
    elif len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][2])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 3 ('{syllables[1][2]}') ของวรรคที่ 2"
        )
    elif len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][3])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 4 ('{syllables[1][3]}') ของวรรคที่ 2"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ❌: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') ไม่สัมผัสกับคำที่ 2 ('{syllables[1][1]}')"
            f"หรือ 3 ('{syllables[1][2]}') หรือ 4 ('{syllables[1][3]}') ของวรรคที่ 2"
        )
```

รูปที่ 5.15 โค้ดตรวจสอบกลอนหก กฎข้อที่ 1

```
if len(syllables[1]) < 4 or len(syllables[2]) < 4:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 2 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 2 สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3)"
    )
else:
    if len(syllables) >= 3 and kv.is_sumpus(syllables[1][-1], syllables[2][-1]):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ✅: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ❌: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
```

รูปที่ 5.16 โค้ดตรวจสอบกลอนหก กฎข้อที่ 2

```python
if len(syllables[2]) < 4 or len(syllables[3]) < 4:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 3 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 3 สัมผัสคล้องจองกับคำที่ 2 หรือ 3 หรือ 4 ของวรรคที่ 4)"
    )
else:
    if len(syllables) >= 4 and \
        (kv.is_sumpus(syllables[2][-1], syllables[3][1])) and \
            (kv.is_sumpus(syllables[2][-1], syllables[3][2])) and \
                (kv.is_sumpus(syllables[2][-1], syllables[3][3])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') ไม่สัมผัสกับคำที่ 2"
            f("{syllables[3][1]}') หรือ 3 ('{syllables[3][2]}') หรือ 4 ('{syllables[3][3]}') ของวรรคที่ 4"
        )
    elif len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
    elif len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][2])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 3 ('{syllables[3][2]}') ของวรรคที่ 4"
        )
    elif len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][3])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 4 ('{syllables[3][3]}') ของวรรคที่ 4"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ❌: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') ไม่สัมผัสกับคำที่ 2"
            f("{syllables[3][1]}') หรือ 3 ('{syllables[3][2]}') หรือ 4 ('{syllables[3][3]}') ของวรรคที่ 4"
        )
```

รูปที่ 5.17 โค้ดตรวจสอบกลอนหก กฎข้อที่ 3

```python
if index < len(klon_list) - 1:  # ตรวจสอบว่าบทถัดไปมีอยู่
    next_klon_lines = klon_list[index + 1].split('\n')
    next_syllables = [syllable_tokenize(line) for line in next_klon_lines]

    if len(syllables[3]) < 4 or len(next_syllables[1]) < 4:
        result['ข้อผิดพลาด'].append(
            f"กฎที่ 4 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 4 ของบท"
            f"{index + 1} สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2})"
        )
    else:
        if len(syllables) >= 4 and len(next_syllables) >= 2:
            if kv.is_sumpus(syllables[3][-1], next_syllables[1][-1]):
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ✅: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
            else:
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ❌: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
```

รูปที่ 5.18 โค้ดตรวจสอบกลอนหก กฎข้อที่ 4

เมื่อทำการตรวจสอบเสร็จสิ้นผลลัพธ์ของฟังก์ชันจะถูกส่งกลับมาในรูปแบบของ List of Dictionary โดยแต่ละ Dictionary จะประกอบด้วย

- "บท": หมายเลขบทที่ตรวจสอบ
- "ผลการตรวจสอบ": รายละเอียดการตรวจสอบ
- "ข้อผิดพลาด": รายละเอียดข้อผิดพลาดที่พบ

การตรวจสอบกลอนเจ็ด

ฟังก์ชัน check_klon7 ทำหน้าที่ในการตรวจสอบกลอนเจ็ด โดยแบ่งการทำงานออกเป็นส่วนต่าง ๆ เพื่อวิเคราะห์บทกลอนและตรวจสอบความถูกต้องตามกฎสัมผัส

ฟังก์ชันนี้จะรับข้อมูลกลอนเจ็ดจากผู้ใช้ในรูปแบบข้อความ (string) แล้วแยกออกเป็นบท ๆ เพื่อตรวจสอบแต่ละบทอย่างละเอียด รวมถึงตรวจสอบข้อผิดพลาดต่าง ๆ ที่อาจเกิดขึ้น เช่น จำนวนวรรคไม่ครบ หรือสัมผัสคล้องจองไม่ถูกต้องตามกฎ

การทำงานของฟังก์ชัน check_klon7 มีขั้นตอนการทำงานดังนี้

1) ระบบจะรับข้อมูลกลอนเจ็ดในรูปแบบของข้อความ (input_klon) ซึ่งผู้ใช้งานสามารถป้อนบทกลอนที่ต้องการตรวจสอบได้หลายบท
2) ระบบทำการแยกบทกลอนตามบรรทัดว่าง (\n\n) เพื่อนำไปตรวจสอบเป็นบท ๆ แต่ละบทจะถูกเก็บในรูปแบบ List ของข้อความ (ดังแสดงในรูปที่ 5.6)
3) ระบบจะทำการแยกวรรคตามบรรทัด (\n) และใช้ฟังก์ชัน syllable_tokenize() ในการแยกพยางค์ของแต่ละวรรคเก็บไว้ในลิสต์ syllables (ดังแสดงในรูปที่ 5.7)
4) ระบบทำการตรวจสอบจำนวนวรรคในแต่ละบท หากบทนั้นมีวรรคไม่ครบ 4 วรรค จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.8)
5) ระบบทำการตรวจสอบจำนวนพยางค์ในแต่ละวรรค แต่ละวรรคต้องมีจำนวนพยางค์ครบ 7 หรือ 8 พยางค์ หากขาดหรือเกิน จะทำการบันทึกข้อผิดพลาด (ดังแสดงในรูปที่ 5.19)

```
for i in range(4):
    if len(syllables[i]) != 7 and len(syllables[i]) != 8:
        result["ข้อผิดพลาด"].append(
            f"วรรคที่ {i+1} ของบท {index+1} จำนวนพยางค์ขาดหรือเกินที่ต้องการ (7 หรือ 8 พยางค์) ผลลัพธ์การตรวจสอบอาจไม่ถูกต้อง"
        )
```

รูปที่ 5.19 โค้ดตรวจสอบจำนวนพยางค์ในแต่ละวรรคในกลอนเจ็ด

6) ตรวจสอบสัมผัสตามกฎกลอนเจ็ด: ฟังก์ชันจะตรวจสอบสัมผัสในแต่ละวรรคตามกฎสัมผัสของกลอนเจ็ด โดยใช้ฟังก์ชัน kv.is_sumpus เพื่อตรวจสอบการสัมผัส โดยการตรวจกลอนเจ็ดมีขั้นตอนการตรวจสอบดังนี้

   กฎที่ 1: คำสุดท้ายของวรรคที่ 1 สัมผัสกับคำที่ 2 ของวรรคที่ 2 (ดังแสดงในรูปที่ 5.20)

   กฎที่ 2: คำสุดท้ายของวรรคที่ 2 สัมผัสกับคำสุดท้ายของวรรคที่ 3 (ดังแสดงในรูปที่ 5.21)

   กฎที่ 3: คำสุดท้ายของวรรคที่ 3 สัมผัสกับคำที่ 2 ของวรรคที่ 4 (ดังแสดงในรูปที่ 5.22)

   กฎที่ 4: คำสุดท้ายของวรรคที่ 4 ของบทปัจจุบันสัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบทถัดไป (ดังแสดงในรูปที่ 5.23)

เมื่อทำการตรวจสอบเสร็จสิ้นผลลัพธ์ของฟังก์ชันจะถูกส่งกลับมาในรูปแบบของ List of Dictionary โดยแต่ละ Dictionary จะประกอบด้วย

- "บท": หมายเลขบทที่ตรวจสอบ
- "ผลการตรวจสอบ": รายละเอียดการตรวจสอบ
- "ข้อผิดพลาด": รายละเอียดข้อผิดพลาดที่พบ

```python
if len(syllables[0]) < 5 or len(syllables[1]) < 5:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 1 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 1 สัมผัสคล้องจองกับคำที่ 2 ของวรรคที่ 2)"
    )
else:
    if len(syllables) >= 2 and (kv.is_sumpus(syllables[0][-1], syllables[1][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ✅: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 1 ❌: คำสุดท้ายของวรรคที่ 1 ('{syllables[0][-1]}') ไม่สัมผัสกับคำที่ 2 ('{syllables[1][1]}') ของวรรคที่ 2"
        )
```

รูปที่ 5.20 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 1

```python
if len(syllables[1]) < 5 or len(syllables[2]) < 5:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 2 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 2 สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3)"
    )
else:
    if len(syllables) >= 3 and kv.is_sumpus(syllables[1][-1], syllables[2][-1]):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ✅: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 2 ❌: คำสุดท้ายของวรรคที่ 2 ('{syllables[1][-1]}') ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}')"
        )
```

รูปที่ 5.21 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 2

```python
if len(syllables[2]) < 5 or len(syllables[3]) < 5:
    result["ข้อผิดพลาด"].append(
        "กฎที่ 3 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 3 สัมผัสคล้องจองกับคำที่ 2 ของวรรคที่ 4)"
    )
else:
    if len(syllables) >= 4 and (kv.is_sumpus(syllables[2][-1], syllables[3][1])):
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ✅: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') สัมผัสคล้องจองกับคำที่ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
    else:
        result["ผลการตรวจสอบ"].append(
            f"กฎที่ 3 ❌: คำสุดท้ายของวรรคที่ 3 ('{syllables[2][-1]}') ไม่สัมผัสกับคำที่ 2 ('{syllables[3][1]}') ของวรรคที่ 4"
        )
```

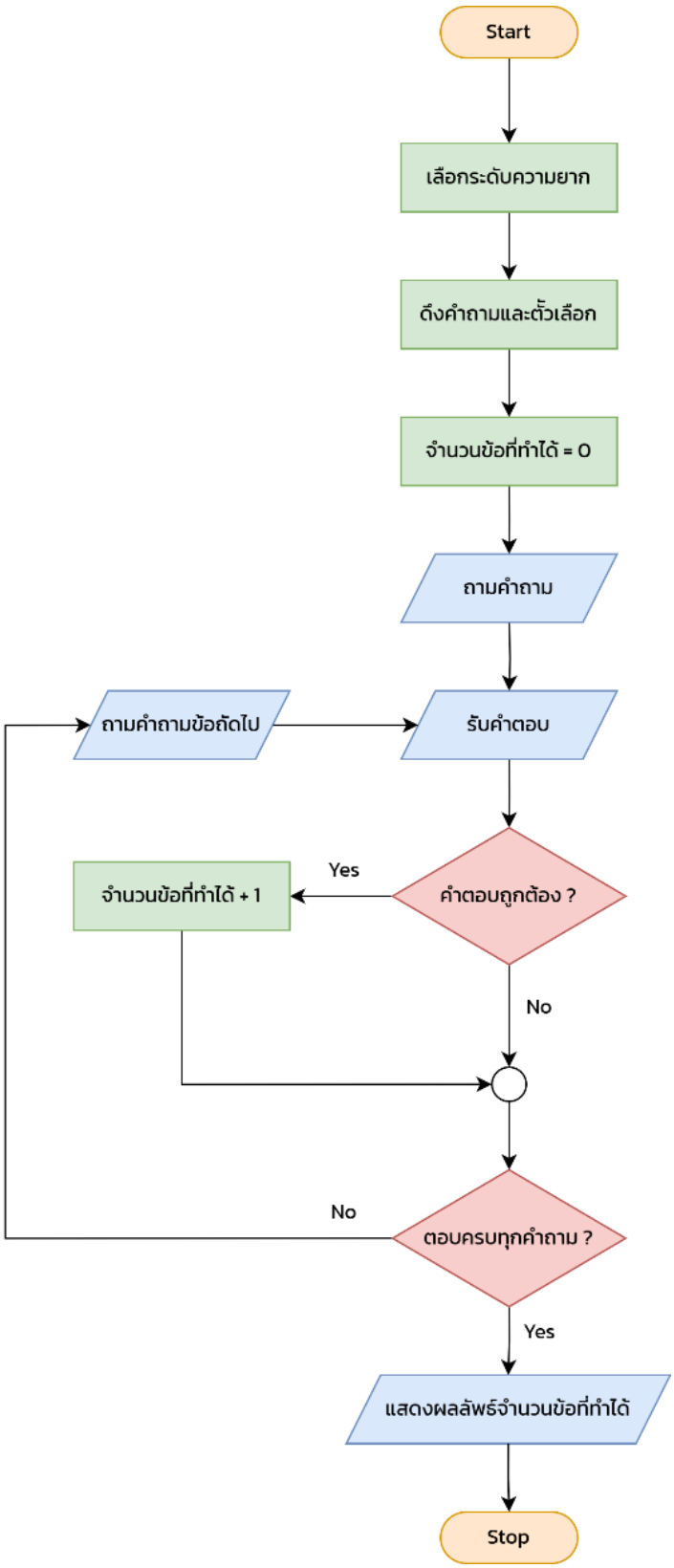
รูปที่ 5.22 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 3

```python
if index < len(klon_list) - 1:  # ตรวจสอบว่าบทถัดไปมีอยู่
    next_klon_lines = klon_list[index + 1].split('\n')
    next_syllables = [syllable_tokenize(line) for line in next_klon_lines]

    if len(syllables[3]) < 5 or len(next_syllables[1]) < 5:
        result['ข้อผิดพลาด'].append(
            f"กฎที่ 4 : ไม่สามารถตรวจสอบได้เนื่องจากจำนวนพยางค์ไม่ถูกต้อง (คำสุดท้ายของวรรคที่ 4 ของบท {index + 1}"
            f"สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2})"
        )
    else:
        if len(syllables) >= 4 and len(next_syllables) >= 2:
            if kv.is_sumpus(syllables[3][-1], next_syllables[1][-1]):
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ✅: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"สัมผัสคล้องจองกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
            else:
                result["ผลการตรวจสอบ"].append(
                    f"กฎที่ 4 ❌: คำสุดท้ายของวรรคที่ 4 ของบท {index + 1} ('{syllables[3][-1]}')"
                    f"ไม่สัมผัสกับคำสุดท้ายของวรรคที่ 2 ของบท {index + 2} ('{next_syllables[1][-1]}')"
                )
```

รูปที่ 5.23 โค้ดตรวจสอบกลอนเจ็ด กฎข้อที่ 4

5.3 การทำงานเกมจับคู่คำคล้องจอง



รูปที่ 5.24 Flowchart การทำงานของเกมจับคู่คำคล้องจอง

ขั้นตอนการทำงานของเกมจับคู่คำคล้องจอง (ดังแสดงในรูปที่ 5.24) มีขั้นตอนดังนี้

1) เริ่มต้นระบบ: ผู้ใช้งานเริ่มต้นการทำงานของระบบ โดยเข้าสู่หน้าเลือกระดับความยากของเกม
2) การเลือกระดับความยาก: ผู้ใช้เลือกระดับความยากของคำถาม โดยแบ่งเป็น ระดับง่าย ระดับปานกลาง และระดับยาก เพื่อปรับเนื้อหาคำถามให้เหมาะสมกับความสามารถ
3) ดึงคำถามและตัวเลือก: ระบบดึงคำถามจากฐานข้อมูลพร้อมกับตัวเลือกคำตอบที่เกี่ยวข้อง เพื่อนำมาแสดงให้ผู้ใช้
4) กำหนดค่าเริ่มต้น: ตั้งค่าตัวแปรสำหรับเก็บจำนวนข้อที่ตอบถูก โดยเริ่มต้นที่ศูนย์
5) การแสดงคำถาม: ระบบแสดงคำถามข้อแรกให้กับผู้ใช้ รวมถึงตัวเลือกคำตอบต่าง ๆ
6) การรับคำตอบจากผู้ใช้: ผู้ใช้เลือกคำตอบที่คิดว่าถูกต้อง จากตัวเลือกที่ระบบให้มา
7) การตรวจสอบคำตอบ: ระบบตรวจสอบคำตอบที่ผู้ใช้เลือกเปรียบเทียบกับคำตอบที่ถูกต้อง หากคำตอบถูกต้อง ระบบจะเพิ่มจำนวนข้อที่ตอบถูกหนึ่งข้อ หากคำตอบไม่ถูกต้อง ระบบจะไม่เพิ่มจำนวนข้อที่ตอบถูก และไปยังคำถามถัดไป
8) การวนซ้ำคำถาม: ระบบวนซ้ำขั้นตอนการแสดงคำถาม รับคำตอบ และตรวจสอบคำตอบ จนกว่าผู้ใช้จะตอบครบทุกคำถาม
9) สรุปผล: เมื่อผู้ใช้ตอบครบทุกคำถาม ระบบจะสรุปจำนวนข้อที่ตอบถูกต้อง และแสดงผลคะแนนหรือผลลัพธ์ที่ได้
10) สิ้นสุดการทำงาน: ระบบจบการทำงานหลังแสดงผล และรอการใช้งานครั้งถัดไป

5.3.1 รายละเอียดการเลือกระดับความยาก

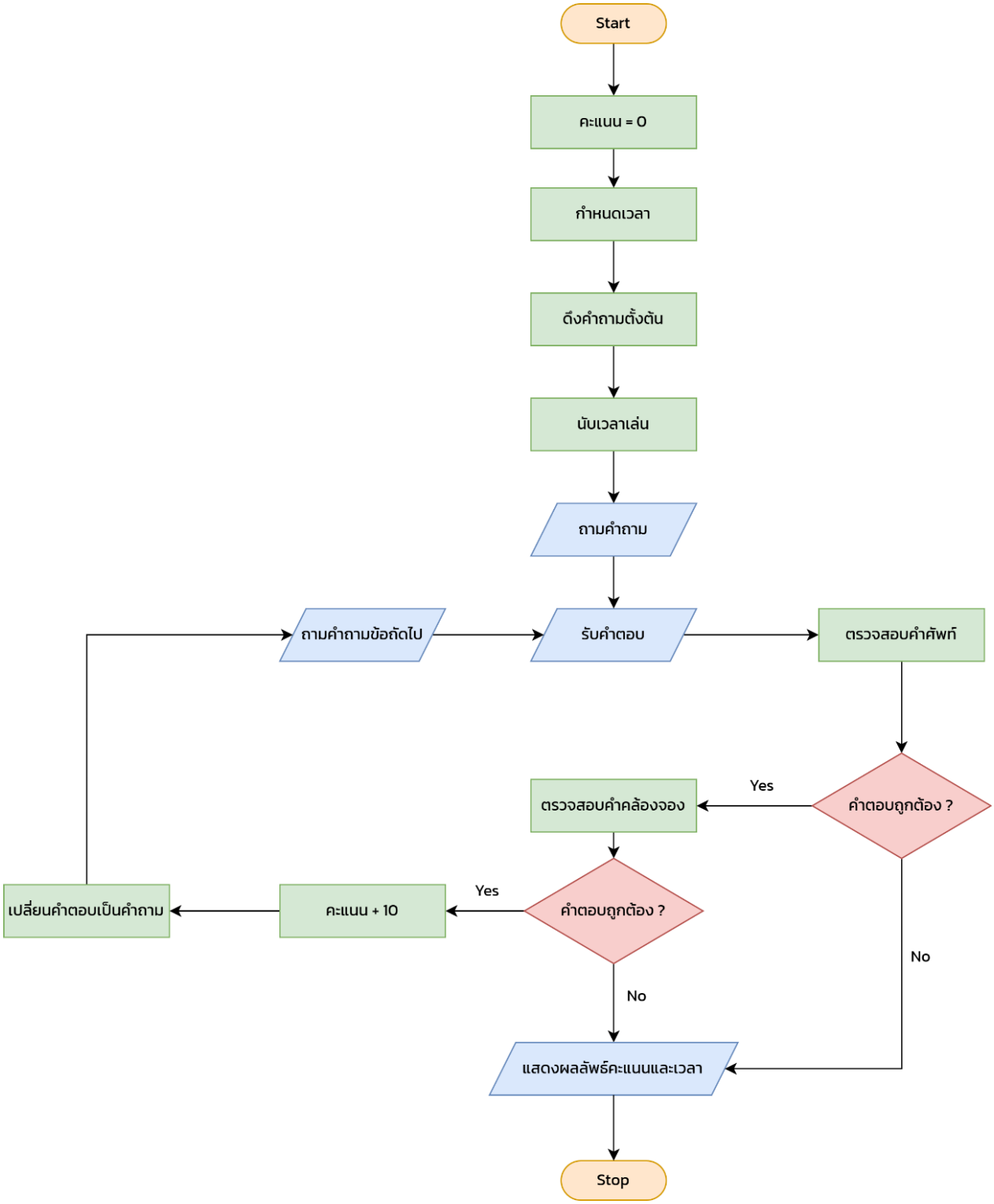
ในระบบเกมจับคู่คำคล้องจอง ผู้ใช้งานสามารถเลือกระดับความยากได้ 3 ระดับ ซึ่งแต่ละระดับจะแตกต่างกันตามความซับซ้อนของคำศัพท์ที่ใช้เป็นคำถามและตัวเลือกคำตอบ โดยรายละเอียดของแต่ละระดับมีดังนี้

1) ระดับง่าย: คำศัพท์ที่ใช้จะมาจากคลังคำสำหรับนักเรียนชั้นประถมศึกษาปีที่ 1 โดยเป็นคำพื้นฐานที่พบได้บ่อยในชีวิตประจำวัน เช่น คำที่มีหนึ่งหรือสองพยางค์ เพื่อให้ผู้เล่นสามารถจับคู่คำคล้องจองได้ง่าย
2) ระดับปานกลาง: ใช้คำศัพท์จากคลังคำสำหรับนักเรียนชั้นประถมศึกษาปีที่ 2 ซึ่งมีความซับซ้อนเพิ่มขึ้น เช่น คำที่มีหลายพยางค์ หรือคำที่มีเสียงคล้ายกันบางส่วน เพื่อเพิ่มความท้าทายให้กับผู้เล่น
3) ระดับยาก: ระบบจะดึงคำศัพท์จากคลังคำของนักเรียนชั้นประถมศึกษาปีที่ 3 ซึ่งเป็นคำที่มีโครงสร้างซับซ้อนมากขึ้น เช่น คำที่มีหลายพยางค์และเสียงที่ใกล้เคียงกัน ทำให้ผู้เล่นต้องใช้ทักษะในการแยกแยะและจับคู่คำคล้องจองมากยิ่งขึ้น

5.4 การทำงานของเกมต่อคำคล้องจอง

ขั้นตอนการทำงานของเกมต่อคำคล้องจอง (ดังแสดงในรูปที่ 5.25) มีขั้นตอนดังนี้

1) เริ่มต้นระบบ: ผู้ใช้งานเริ่มต้นการทำงานของระบบโดยเข้าสู่เกมต่อคำคล้องจอง ซึ่งเป็นจุดเริ่มต้นของกระบวนการทั้งหมด
2) กำหนดคะแนน: ระบบกำหนดให้คะแนนเริ่มต้นเป็นศูนย์ เพื่อเตรียมพร้อมสำหรับการคำนวณคะแนนในระหว่างการเล่นเกม
3) กำหนดเวลา: ระบบตั้งเวลาสำหรับการเล่นเกมต่อคำคล้องจอง เพื่อใช้ในการนับเวลาที่เล่น
4) ดึงคำถามตั้งต้น: ระบบดึงคำถามข้อแรกจากฐานข้อมูลหรือจากชุดคำถามที่เตรียมไว้ล่วงหน้า เพื่อแสดงให้ใช้เริ่มต้นตอบคำถาม
5) นับเวลาเล่น: ระบบเริ่มจับเวลาทันทีหลังจากแสดงคำถามข้อแรก เพื่อนับระยะเวลาทั้งหมดที่ผู้ใช้งานใช้ในการเล่นเกม
6) ถามคำถาม: ระบบแสดงคำถามให้กับผู้เล่น โดยคำถามที่แสดงจะเกี่ยวข้องกับการทดสอบความสามารถในการต่อคำคล้องจอง
7) รับคำตอบ: ระบบรับคำตอบจากผู้เล่น หลังจากที่ผู้เล่นกรอกหรือป้อนข้อมูลคำตอบที่คิดว่าเหมาะสมกับคำถามที่ได้รับ
8) ตรวจสอบความถูกต้องของคำศัพท์: ระบบตรวจสอบว่าคำตอบที่ผู้เล่นกรอกมีอยู่ในฐานข้อมูลคำศัพท์ PyThaiNLP หรือไม่ หากคำตอบไม่มีอยู่ในฐานข้อมูล ระบบจะแสดงผลลัพธ์ของเกม โดยสรุปคะแนนรวมและระยะเวลาที่ใช้ในการเล่นเกม จากนั้นสิ้นสุดการทำงาน หากคำตอบมีอยู่ในฐานข้อมูล ระบบจะดำเนินการในขั้นตอนถัดไป
9) ตรวจสอบคำคล้องจอง: ระบบทำการตรวจสอบคำตอบของผู้เล่น โดยเปรียบเทียบกับคำถามเพื่อดูว่าคำตอบนั้นคล้องจองตามเกณฑ์ที่กำหนดหรือไม่ โดยการใช้ระบบตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ
10) ตรวจสอบผลการตอบคำถาม: หากคำตอบถูกต้อง ระบบจะเพิ่มคะแนนให้ผู้เล่นจำนวน 10 คะแนน และนำคำตอบที่ถูกต้องมาใช้เป็นคำถามใหม่ในรอบถัดไป จากนั้นวนกลับไปยังขั้นตอนการแสดงคำถามอีกครั้ง หากคำตอบไม่ถูกต้อง ระบบจะแสดงผลลัพธ์ของเกม โดยสรุปคะแนนรวมที่ผู้เล่นทำได้ พร้อมทั้งแสดงระยะเวลาที่ใช้ในการเล่นเกมทั้งหมด และสิ้นสุดกระบวนการทำงานของระบบ



รูปที่ 5.25 Flowchart การทำงานของเกมต่อคำคล้องจอง

5.5 การทำงานของระบบแปลงเสียงเป็นข้อความ

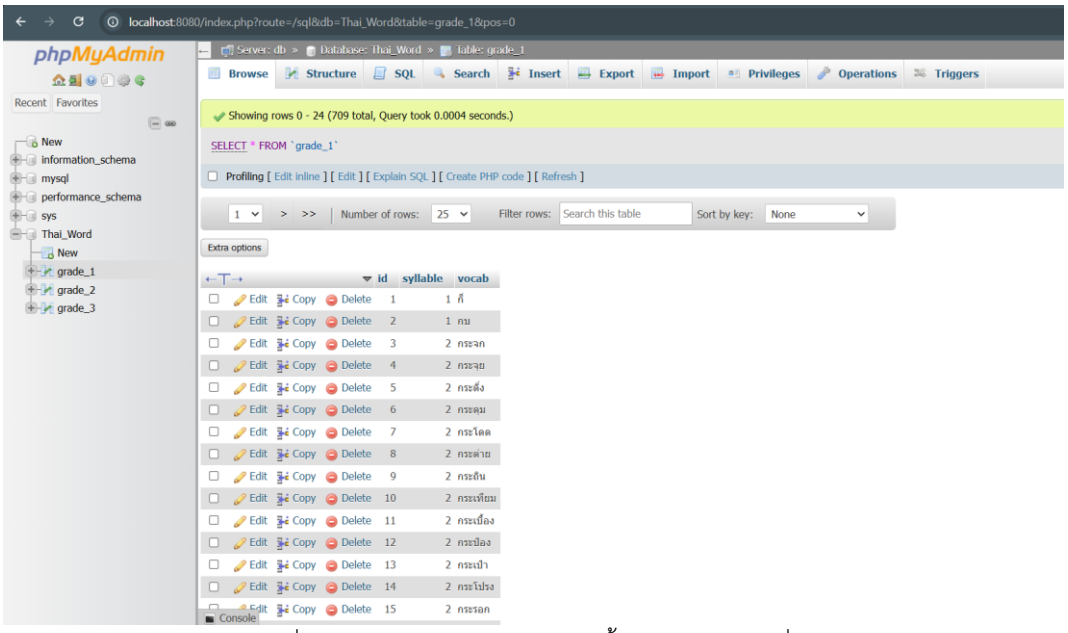
ระบบแปลงเสียงเป็นข้อความ (Speech-to-Text) ในเว็บแอปพลิเคชันถูกออกแบบมาเพื่อรับเสียงจากผู้ใช้งานและแปลงข้อมูลเสียงนั้นให้เป็นข้อความที่สามารถนำไปใช้งานต่อได้ในระบบ โดยมีขั้นตอนการทำงานดังนี้

1) ระบบเริ่มต้นด้วยการเปิดใช้งานไมโครโฟนของอุปกรณ์เพื่อรับเสียงจากผู้ใช้งาน ซึ่งเสียงที่ได้รับจะถูกเก็บเป็นข้อมูลดิจิทัลและส่งไปยังส่วนประมวลผล
2) ข้อมูลเสียงที่ได้รับจะถูกแปลงจากสัญญาณเสียงอนาล็อกเป็นข้อมูลดิจิทัล และทำการวิเคราะห์รูปแบบของเสียง (Audio Signal Processing)
3) หลังจากวิเคราะห์เสียงแล้ว ระบบจะใช้เทคโนโลยีการรู้จำเสียงพูด (Speech Recognition) เช่น Web Speech API หรือไลบรารีที่เกี่ยวข้องในการแปลงข้อมูลเสียงเป็นข้อความที่มีความแม่นยำสูง (ดังแสดงในหัวข้อที่ 3.1)
4) ข้อความที่ได้จากการประมวลผลจะถูกแสดงผลใน Interface ของเว็บแอปพลิเคชันเพื่อให้ผู้ใช้งานสามารถตรวจสอบและนำไปใช้งานต่อได้

5.6 ส่วนของฐานข้อมูล

ในส่วนของฐานข้อมูล จะใช้ในการจัดเก็บคำศัพท์ภาษาไทยสำหรับระดับชั้นประถมศึกษาปีที่ 1 ถึงปีที่ 3 โดยข้อมูลคำศัพท์แต่ละคำประกอบด้วย คำศัพท์ หมายเลขคำศัพท์ และจำนวนพยางค์ ส่วนจำนวนคำศัพท์ของแต่ละชั้นปีมีดังนี้

1) คำศัพท์สำหรับประถมศึกษาปีที่ 1 จำนวน 709 คำ
2) คำศัพท์สำหรับประถมศึกษาปีที่ 2 จำนวน 1,098 คำ
3) คำศัพท์สำหรับประถมศึกษาปีที่ 3 จำนวน 1,210 คำ



รูปที่ 5.26 ตัวอย่างชุดข้อมูลคำศัพท์ชั้นประถมศึกษาปีที่ 1

บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

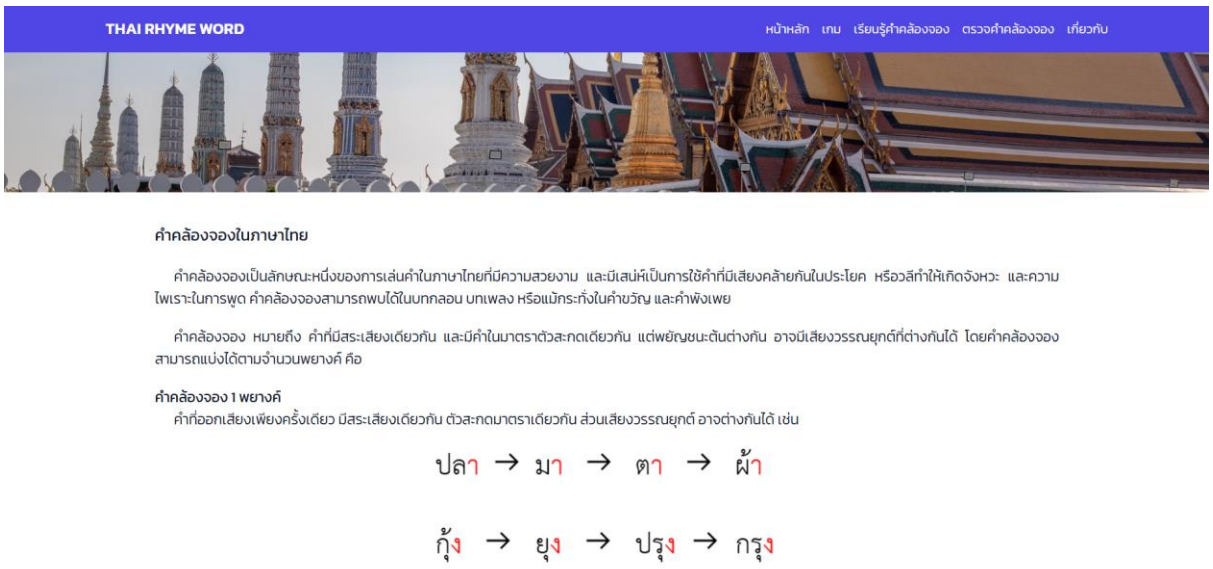
ในบทที่ 6 นี้จะอธิบายถึงความก้าวหน้าผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะจากการทำโครงงานปริญญานิพนธ์ในช่วงระยะเวลาที่ผ่านมา

6.1 ผลการดำเนินงาน

สำหรับผลการดำเนินงานของโครงงานปริญญานิพนธ์เรื่อง “ระบบวิเคราะห์ความคล้องจองของคำภาษาไทย” สามารถทำงานได้ ดังนี้

6.1.1 สื่อการเรียนการสอนเบื้องต้นเกี่ยวกับคำคล้องจองภาษาไทย

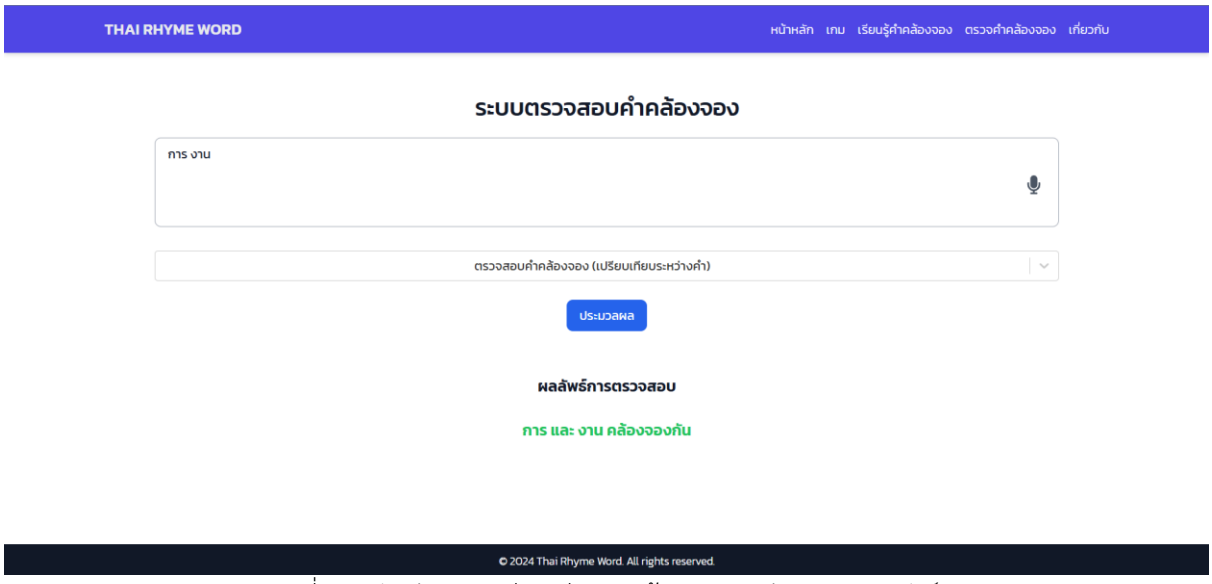
ในส่วนของสื่อการเรียนการสอนเบื้องต้นเกี่ยวกับคำคล้องจองภาษาไทย ได้มีการจัดทำสื่อการเรียนการสอนที่มุ่งเน้นการเสริมสร้างความเข้าใจเกี่ยวกับคำคล้องจองในภาษาไทย โดยเนื้อหาประกอบด้วยข้อมูลพื้นฐานเกี่ยวกับคำคล้องจอง เช่น ความหมายของคำคล้องจอง และตัวอย่างคำคล้องจอง (ดังแสดงในรูปที่ 6.1) เพื่อช่วยให้งานสามารถเข้าใจและนำความรู้ไปประยุกต์ใช้ได้อย่างมีประสิทธิภาพ



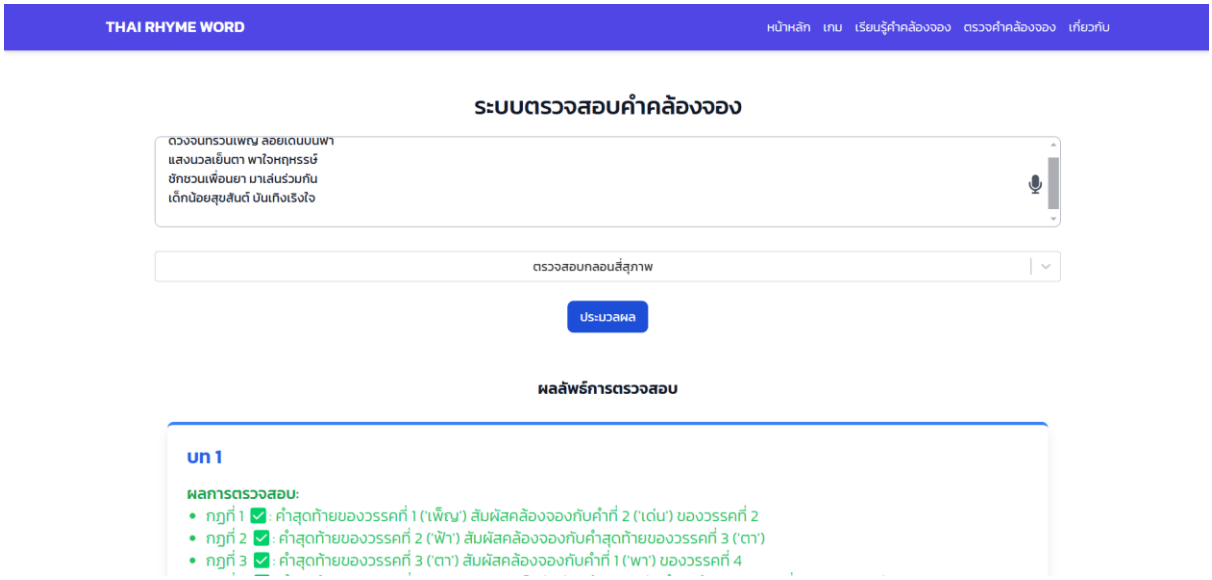
รูปที่ 6.1 สื่อการเรียนการสอนเบื้องต้นเกี่ยวกับคำคล้องจองภาษาไทย

6.1.2 ตรวจสอบคำคล้องจอง

ในขั้นตอนการพัฒนาระบบตรวจสอบคำคล้องจอง ระบบได้รับการออกแบบให้รองรับการตรวจสอบคำคล้องจองได้ 4 วิธี ได้แก่ การตรวจสอบกลอนสี่ กลอนหก และกลอนเจ็ด ซึ่งสามารถวิเคราะห์ความคล้องจองในบทกลอนแต่ละประเภทตามหลักการแต่งกลอนภาษาไทยที่กำหนดไว้ นอกจากนี้ ระบบยังสามารถตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ เพื่อให้ผู้ใช้งานสามารถตรวจสอบคำสองคำว่าคล้องจองกันหรือไม่ โดยการทำงานของระบบสามารถวิเคราะห์และแสดงผลลัพธ์ได้ (ดังแสดงในรูปที่ 6.2 และ 6.3)



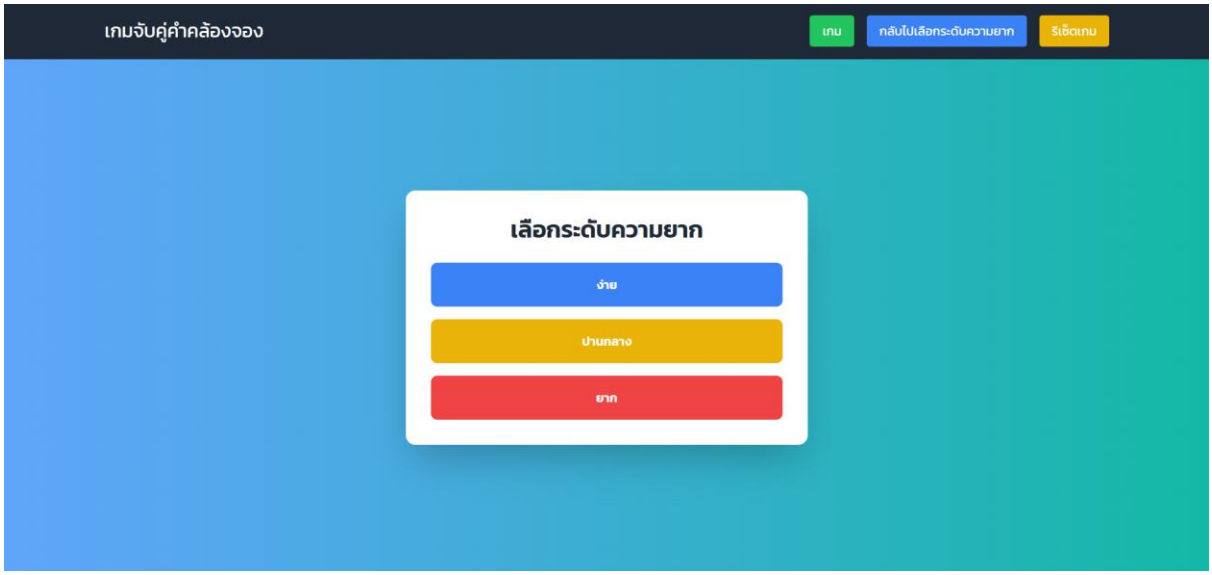
รูปที่ 6.2 ตัวอย่างการเปรียบเทียบคำคล้องจองระหว่างคำและผลลัพธ์



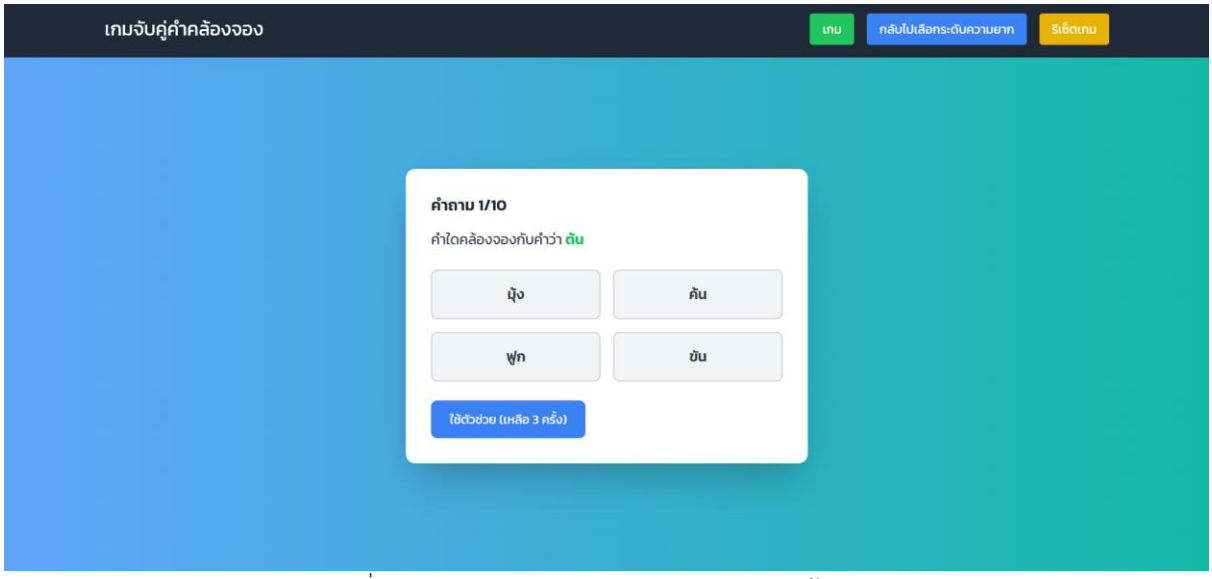
รูปที่ 6.3 ตัวอย่างการตรวจสอบคำคล้องจองในกลอนทุกประเภทและผลลัพธ์

6.1.3 เกมจับคู่คำคล้องจอง

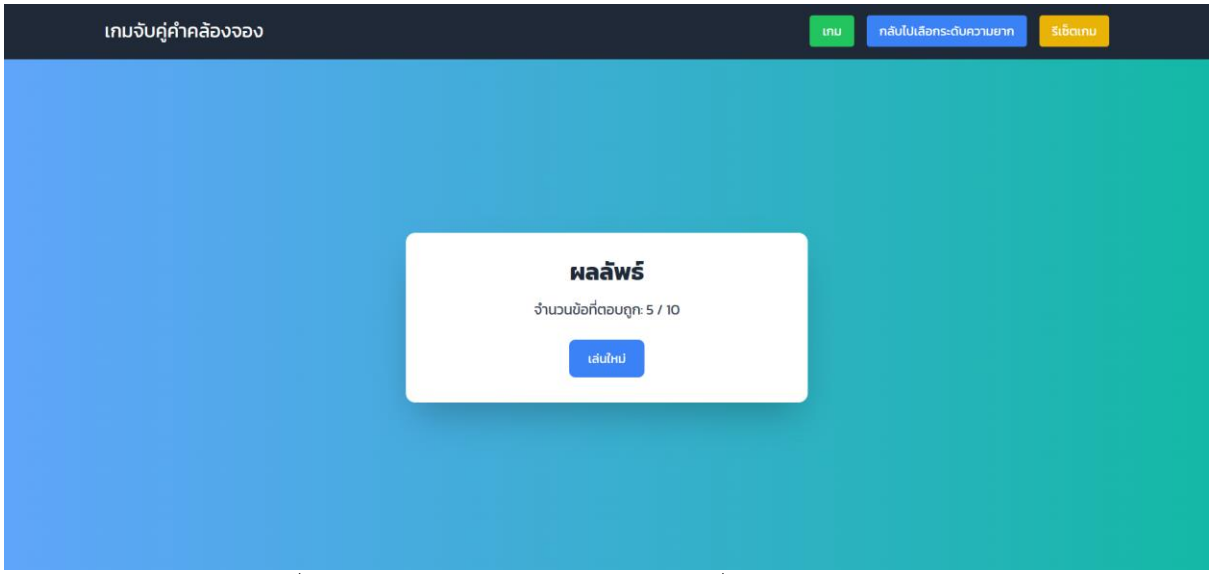
ในส่วนของการพัฒนาและดำเนินการเกมจับคู่คำคล้องจอง ระบบได้รับการออกแบบเพื่อให้ผู้เล่นสามารถฝึกทักษะการจับคู่คำที่มีเสียงคล้องจองกัน โดยระบบจะทำการสุ่มคำถามและตัวเลือกคำตอบจากฐานข้อมูลผ่านทาง API ตามระดับความยากที่ผู้ใช้เลือก (ดังแสดงในรูปที่ 6.4) หลังจากทำการสุ่มสำเร็จระบบจะทำการแสดงคำถามพร้อมตัวเลือกคำตอบให้กับผู้เล่น (ดังแสดงในรูปที่ 6.5) ผู้เล่นสามารถเลือกคำตอบที่คิดว่าเสียงคล้องจองกัน หลังจากเลือกคำตอบ ระบบจะทำการตรวจสอบความถูกต้องของคำตอบที่เลือก พร้อมให้คะแนนตามผลลัพธ์ที่ได้ เมื่อผู้เล่นตอบคำถามครบทุกข้อ ระบบจะทำการแสดงผลลัพธ์ (ดังแสดงในรูปที่ 6.6) นอกจากนี้ เกมยังมีปุ่มตัวช่วยที่สามารถแสดงคำใบ้หรือคำแนะนำให้กับผู้เล่นเมื่อประสบปัญหาในการเลือกคำตอบ รวมถึงปุ่มรีเซ็ตเกมและปุ่มเล่นเกมใหม่ที่ช่วยให้ผู้เล่นสามารถเริ่มต้นเกมใหม่หรือรีเซ็ตสถานะเกมได้ตามที่ต้องการ



รูปที่ 6.4 ระดับความยากเกมจับคู่คำคล้องจอง



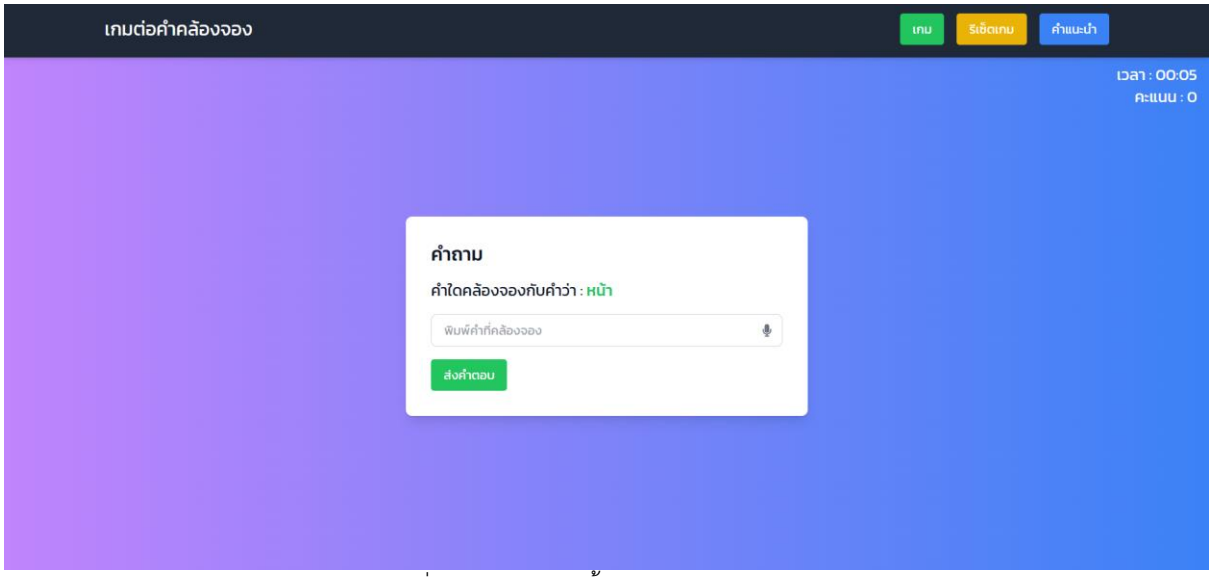
รูปที่ 6.5 ตัวอย่างคำถามและคำตอบเกมจับคู่คำคล้องจอง



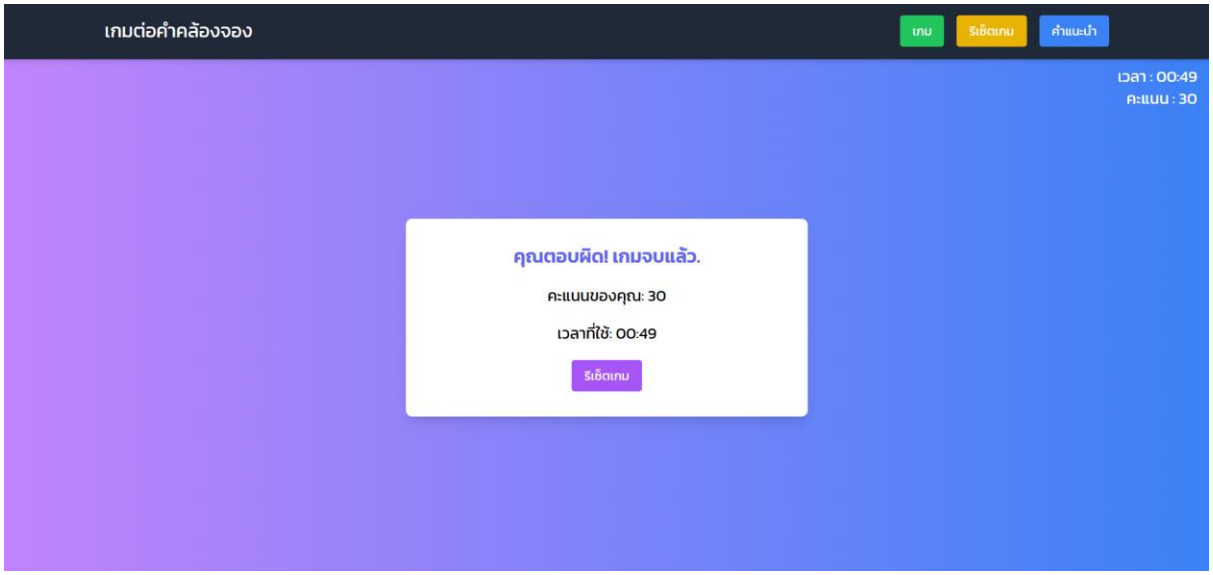
รูปที่ 6.6 ตัวอย่างผลลัพธ์เกมจับคู่คำคล้องจองเมื่อตอบคำถามครบทุกข้อ

6.1.4 เกมต่อคำคล้องจอง

ในส่วนของการพัฒนาเกมต่อคำคล้องจอง ระบบได้รับการออกแบบเพื่อให้ผู้ใช้งานสามารถฝึกทักษะการจับคู่คำที่คล้องจอง โดยเริ่มต้นจากการสุ่มคำตั้งต้นจากฐานข้อมูลผ่านทาง API และแสดงคำถามให้ผู้ใช้งาน (ดังแสดงในรูปที่ 6.7) เมื่อผู้ใช้งานตอบคำที่คล้องจองกับคำตั้งต้นอย่างถูกต้อง ระบบจะนำคำที่ผู้ใช้ตอบมาเป็นคำถามต่อไป และให้ผู้ใช้ตอบคำที่คล้องจองกับคำถามนั้น ๆ กระบวนการนี้จะดำเนินไปจนกว่าผู้ใช้จะตอบคำที่ไม่คล้องจอง เมื่อคำตอบไม่คล้องจอง ระบบจะทำการแสดงผลลัพธ์เป็นเวลาและคะแนนที่ทำได้ (ดังแสดงในรูปที่ 6.8)



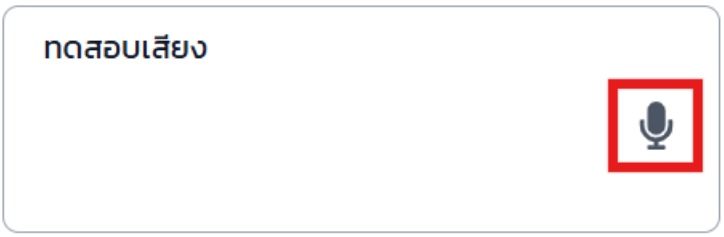
รูปที่ 6.7 ตัวอย่างคำตั้งต้นเกมต่อคำคล้องจอง



รูปที่ 6.8 ตัวอย่างผลลัพธ์เกมต่อคำคล้องจองเมื่อคำตอบไม่คล้องจองกับคำถาม

6.1.5 การแปลงข้อมูลเสียงเป็นข้อความ

ในการพัฒนาระบบแปลงเสียงเป็นข้อความ ระบบได้รับการออกแบบให้สามารถรับเสียงจากไมโครโฟนของผู้ใช้และแปลงเสียงเหล่านั้นเป็นข้อความ โดยใช้เทคโนโลยีการรู้จำเสียงพูด (Speech Recognition) เมื่อผู้ใช้พูด ระบบจะทำการแปลงคำพูดเป็นข้อความและแสดงผลลัพธ์ (ดังแสดงในรูปที่ 6.9)



รูปที่ 6.9 ตัวอย่างระบบแปลงเสียงเป็นข้อความ

6.2 ข้อจำกัด

1) ในภาษาไทยมีคำที่มีการเขียนและการออกเสียงที่หลากหลาย ซึ่งบางคำมีความยากต่อการวิเคราะห์ได้อย่างแม่นยำ โดยเฉพาะคำที่เป็นพหุพยางค์หรือคำที่เกิดจากการรวมคำหลาย ๆ คำ (คำสมาส, คำสนธิ) เช่น คำที่ยืมจากภาษาบาลี และสันสกฤต
2) การลดรูปของคำสมาสและคำสนธิ: คำที่มาจากการรวมคำ (สมาส/สนธิ) เช่น "ราชการ" มาจาก "ราช" และ "การ" อาจทำให้ PyThaiNLP ไม่สามารถตัดคำหรือแบ่งพยางค์ได้ถูกต้อง เพราะการลดรูปเสียงไม่ได้สะท้อนในรูปเขียน

3) คำที่เขียนไม่ตรงกับการออกเสียง: คำบางคำในภาษาไทยมีการเขียนที่ไม่สอดคล้องกับการออกเสียง เช่น "ราชการ" ซึ่งเขียนแบบหนึ่ง (ราช-การ) แต่มีการออกเสียงเป็นอีกแบบ (ราด-ชะ-กาน) PyThaiNLP ใช้การตัดคำตามตัวอักษรที่ปรากฏ ทำให้การตัดพยางค์อาจไม่ตรงกับการออกเสียงจริง

6.3 ข้อเสนอแนะ

1) ปรับปรุงวิธีการตัดคำและตัดพยางค์ของไลบรารี่ PyThaiNLP เพื่อให้การตัดคำและพยางค์ในภาษาไทยมีความถูกต้องและแม่นยำมากยิ่งขึ้น
2) ระบบตรวจสอบคำคล้องจองสามารถพัฒนาไปเป็นเกมการแต่งคำคล้องจองได้
3) เพิ่มระบบตรวจสอบคำคล้องจองประเภทที่ไม่ใช่กลอนเพื่อใช้ประโยชน์ในด้านอื่น ๆ เช่น การแต่งเพลง เมื่อมีระบบนี้ผู้แต่งสามารถเห็นภาพรวมของเนื้อเพลงได้ชัดเจน ทำให้ผู้แต่งสามารถแต่งเนื้อเพลงให้ลื่นไหลได้ดีขึ้น

บรรณานุกรม

[1] พระยาอุปกิตศิลปสาร, ไวยากรณ์ไทย อักขรวิธี, ไทยวัฒนาพานิช, 2492, น. 3-6.

[2] อ. แสงอัสนีย์ และ ส. บุญอุรพีภิญโญ, ชุดกิจกรรมการเรียนรู้พัฒนาการคิดวิเคราะห์ ภาษาไทย ป.3, บริษัท สถาบันพัฒนาคุณภาพวิชาการ (พว.) จำกัด, น. 28, 36.

[3] อ. แสงอัสนีย์ และ ส. บุญอุรพีภิญโญ, ชุดกิจกรรมการเรียนรู้พัฒนาการคิดวิเคราะห์ ภาษาไทย ป.3, บริษัท สถาบันพัฒนาคุณภาพวิชาการ (พว.) จำกัด, น. 137-138.

[4] ช. เรืองลายคราม, "Tailwind CSS: A framework that makes dev work easier," [Online]. Available: https://morphos.is/th/blog/tailwind-css-a-framework-that-makes-dev-work-easier.

[5] A. Pickering, "Shiritori," 2018. [Online]. Available: https://shiritorigame.com/. [Accessed 12 พฤษภาคม 2567].

[6] Amazon Web Services, "Speech to Text คืออะไร," [Online]. Available: https://aws.amazon.com/th/what-is/speech-to-text/. [Accessed 14 พฤษภาคม 2567].

[7] Amazon Web Services, "Docker คืออะไร," [Online]. Available: https://aws.amazon.com/th/docker/. [Accessed 14 พฤษภาคม 2567].

[8] flywithken, "RHYMETHAI," 2018. [Online]. Available: https://rhymethai.com/. [Accessed 8 พฤษภาคม 2567].

[9] M. Lin, "B-RHYMES," 2024. [Online]. Available: http://www.b-rhymes.com/. [Accessed 12 พฤษภาคม 2567].

[10] P. Saadeghi, "What is daisyUI?," [Online]. Available: https://daisyui.com/blog/what-is-daisyui/.

[11] PythaiNLP Team, "FAQ," [Online]. Available: https://pythainlp.github.io/FAQ. [Accessed 15 พฤษภาคม 2567].

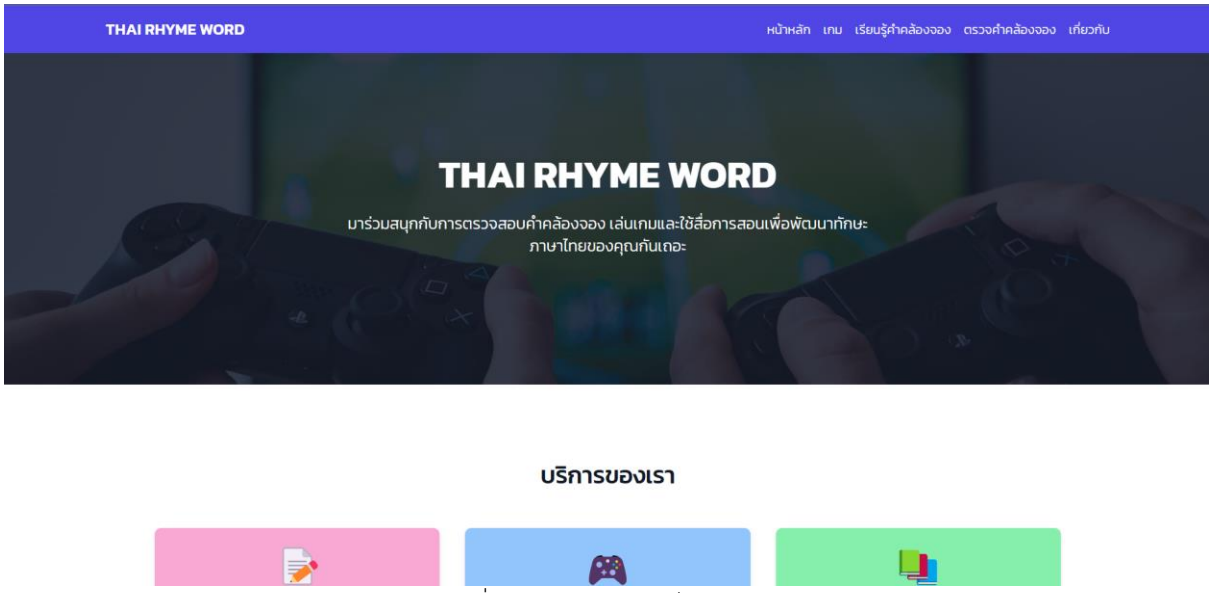
[12] R. Deysel, "Rhymey," [Online]. Available: https://chromewebstore.google.com/detail/rhymey/fbkmdcolngnmmhmdkhngfmdmeofipahp?hl=es. [Accessed 12 พฤษภาคม 2567].

[13] WriteExpress LLC, "Rhymer," WriteExpress LLC, 1996. [Online]. Available: https://www.rhymer.com/. [Accessed 12 พฤษภาคม 2567].

ภาคผนวก ก คู่มือการใช้งาน

ก.1 หน้าแรกของเว็บแอปพลิเคชัน

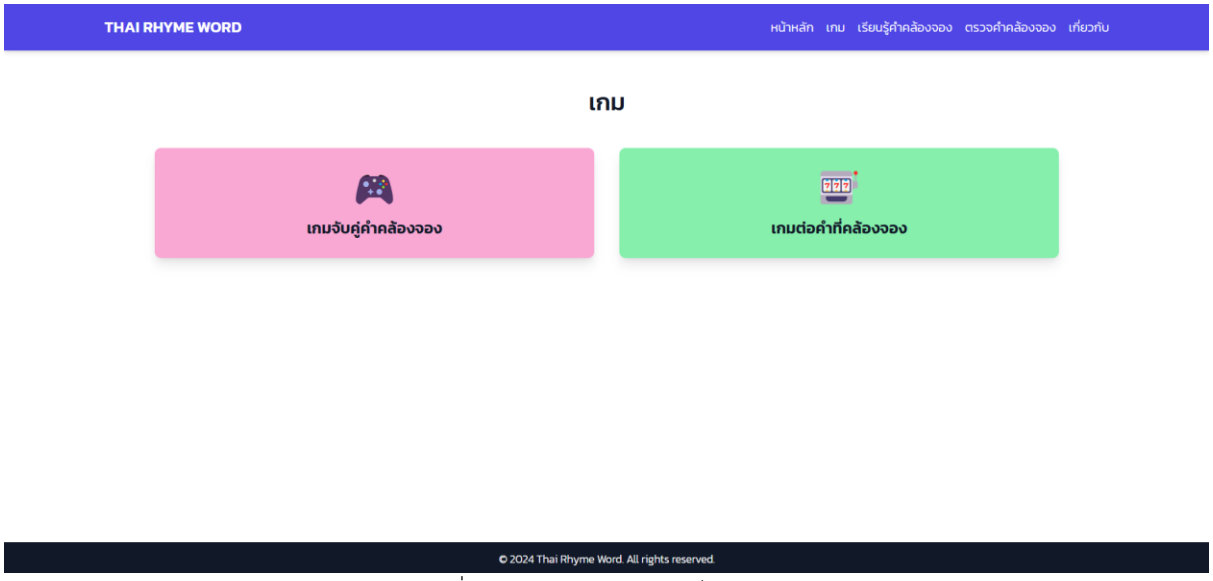
เมื่อผู้ใช้งานเข้ามาใช้งานเว็บแอปพลิเคชัน หน้าแรกที่จะแสดงให้เห็นก็คือหน้าแรกของเว็บไซต์ที่ใช้ในการแนะนำเว็บไซต์ และบริการที่มีอยู่ (ดังแสดงในรูปผนวกที่ ก.1)



รูปผนวกที่ ก.1 หน้าแรกของเว็บแอปพลิเคชัน

ก.2 หน้าเกม

หน้าเกมจะเป็นหน้าที่ใช้แสดงเกมที่มีอยู่ในเว็บแอปพลิเคชัน โดยผู้ใช้สามารถเลือกเล่นได้ 2 เกม คือ เกมจับคู่คำคล้องจอง และ เกมต่อคำคล้องจอง (ดังแสดงในรูปผนวกที่ ก.2)



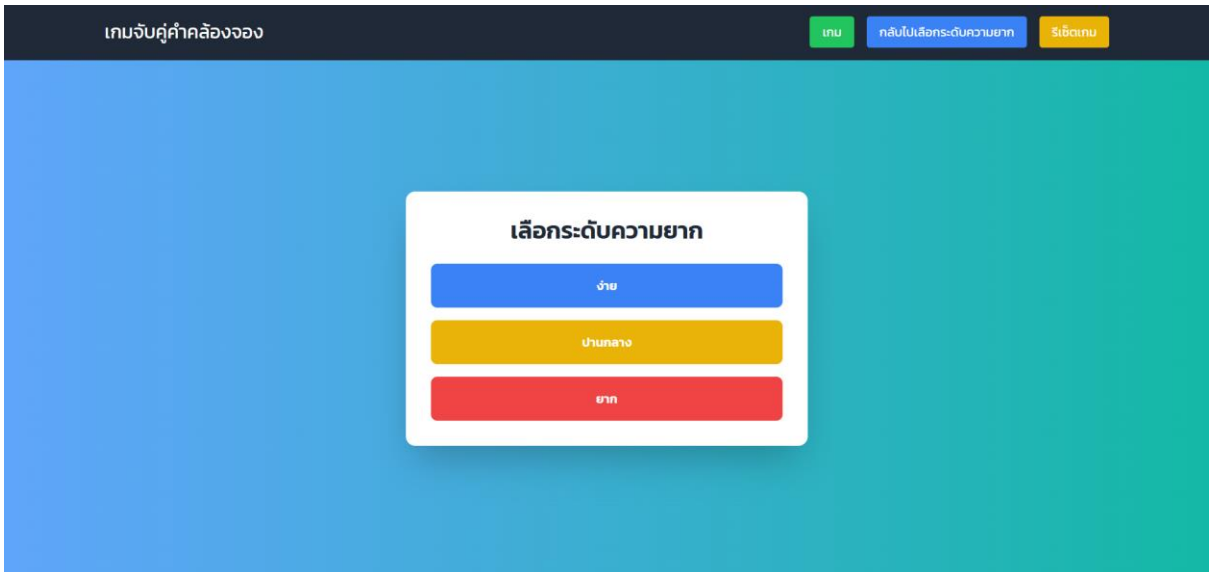
รูปผนวกที่ ก.2 หน้าเลือกเกมของเว็บแอปพลิเคชัน

ก.2.1 เกมจับคู่คำคล้องจอง

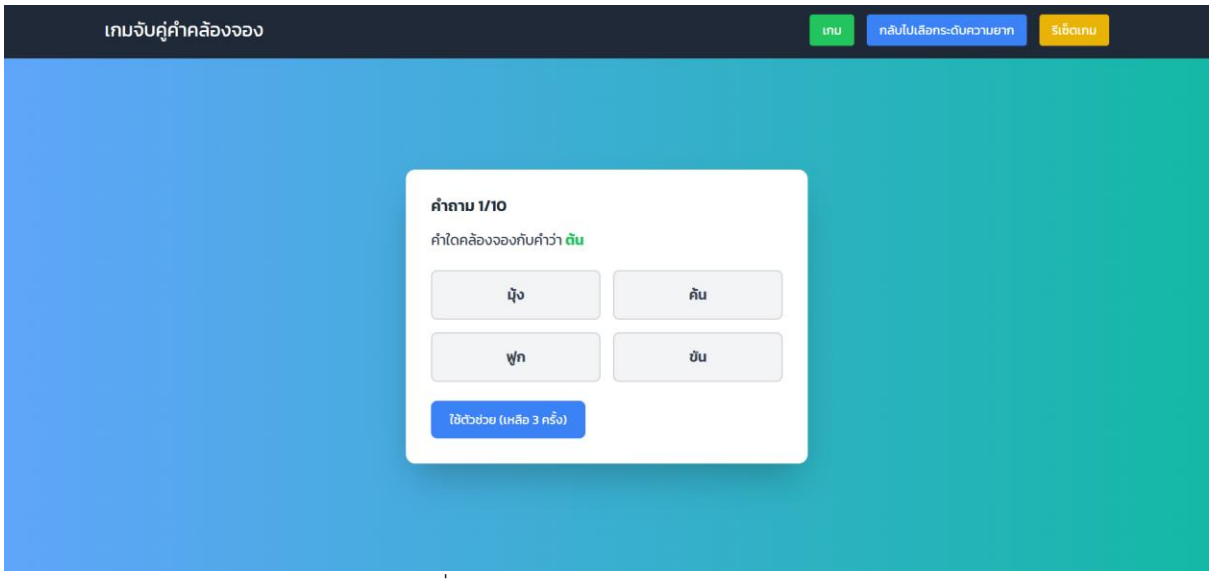
เกมจับคู่คำคล้องจองเป็นเกมที่ผู้เล่นต้องเลือกคำตอบที่คล้องจองกับคำถามให้ถูกต้อง โดยจะมีการนับคะแนนเมื่อผู้เล่นตอบถูก และจะไม่มีคะแนนหากตอบผิด เกมนี้ถูกออกแบบให้มี 3 ระดับ ดังนี้

1) ระดับง่าย: ใช้คำศัพท์สำหรับเด็กชั้นประถมศึกษาปีที่ 1
2) ระดับปานกลาง: ใช้คำศัพท์สำหรับเด็กชั้นประถมศึกษาปีที่ 2
3) ระดับยาก: ใช้คำศัพท์สำหรับเด็กชั้นประถมศึกษาปีที่ 3

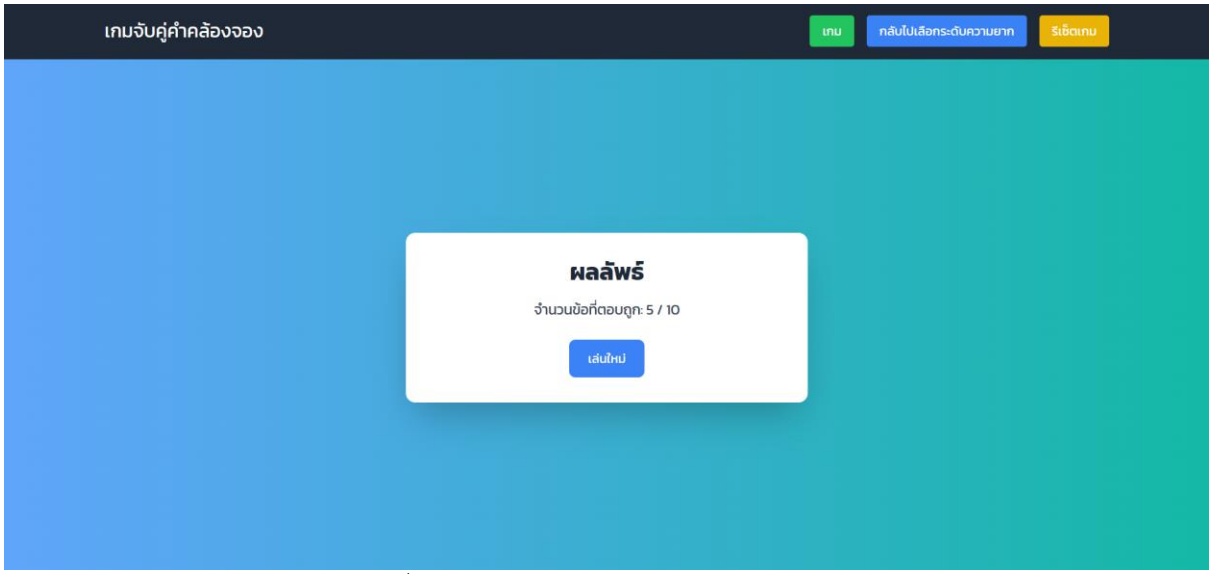
ในเกมมีปุ่มควบคุมต่าง ๆ ได้แก่ ปุ่มกลับหน้าเลือกเกม ปุ่มกลับไปเลือกระดับความยาก และปุ่มรีเซ็ตเกม เกมนี้ยังมีปุ่มช่วยเหลือที่สามารถกดใช้งานได้ 3 ครั้งต่อเกม เมื่อเกมจบ ระบบจะแสดงผลลัพธ์พร้อมปุ่มให้เล่นใหม่อีกครั้ง (ดังแสดงในรูปผนวกที่ ก.3 รูปผนวกที่ ก.4 และรูปผนวกที่ ก.5)



รูปผนวกที่ ก.3 หน้าเลือกระดับความยากของเกมจับคู่คำคล้องจอง



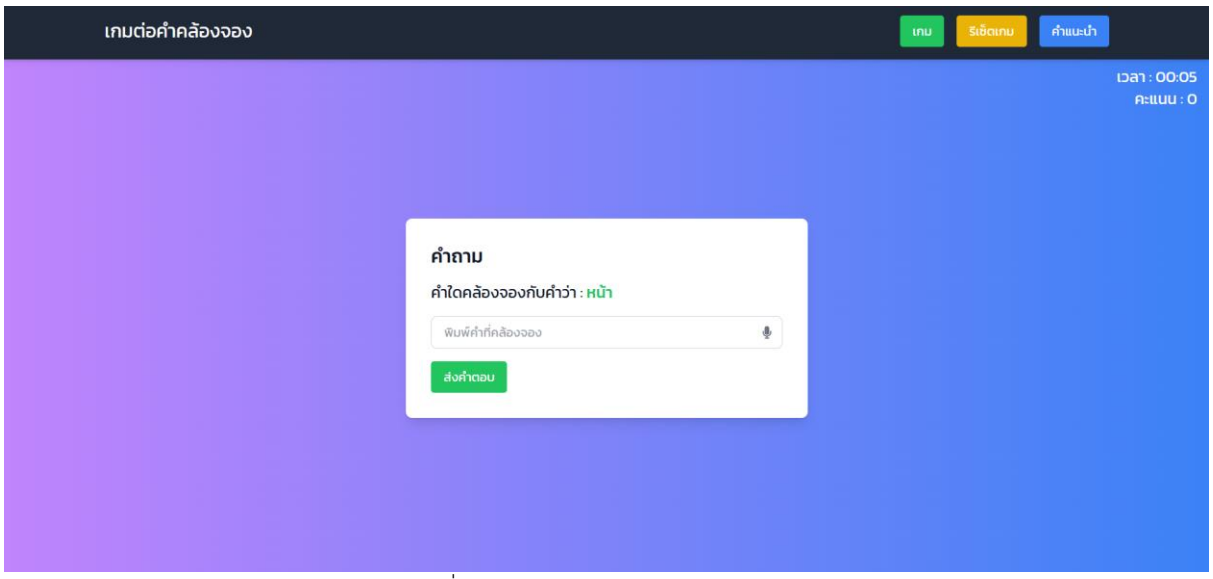
รูปผนวกที่ ก.4 หน้าเกมจับคู่คำคล้องจองขณะเล่นเกม



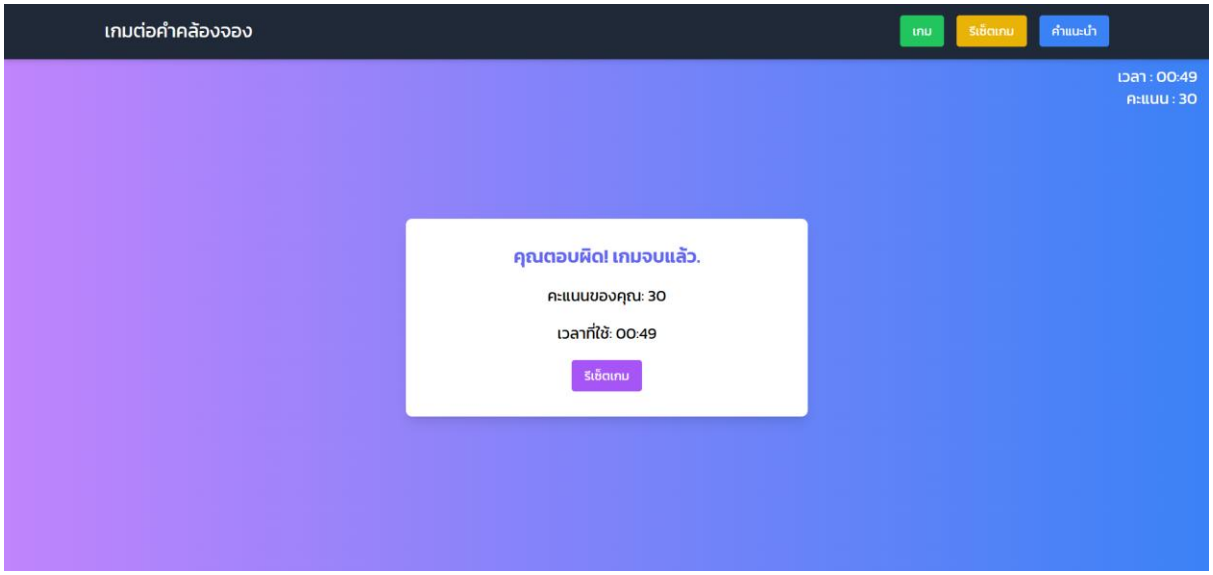
รูปผนวกที่ ก.5 หน้าแสดงผลลัพธ์หลังจบเกมจับคู่คำคล้องจอง

ก.2.2 เกมต่อคำคล้องจอง

เกมต่อคำคล้องจองเป็นเกมที่ผู้เล่นต้องตอบคำที่คล้องจองกับคำเริ่มต้นที่ระบบกำหนด โดยทุกคำตอบจะต้องคล้องจองกับคำที่มาก่อนหน้า หากตอบถูกจะได้คะแนนสะสม แต่หากตอบผิดเกมจะจบลง และแสดงผลลัพธ์พร้อมปุ่มให้เล่นใหม่อีกครั้ง อีกทั้งในเกมมีปุ่มควบคุมต่าง ๆ ได้แก่ ปุ่มกลับหน้าเลือกเกม ปุ่มรีเซ็ตเกม และปุ่มให้คำแนะนำแก่ผู้เล่น



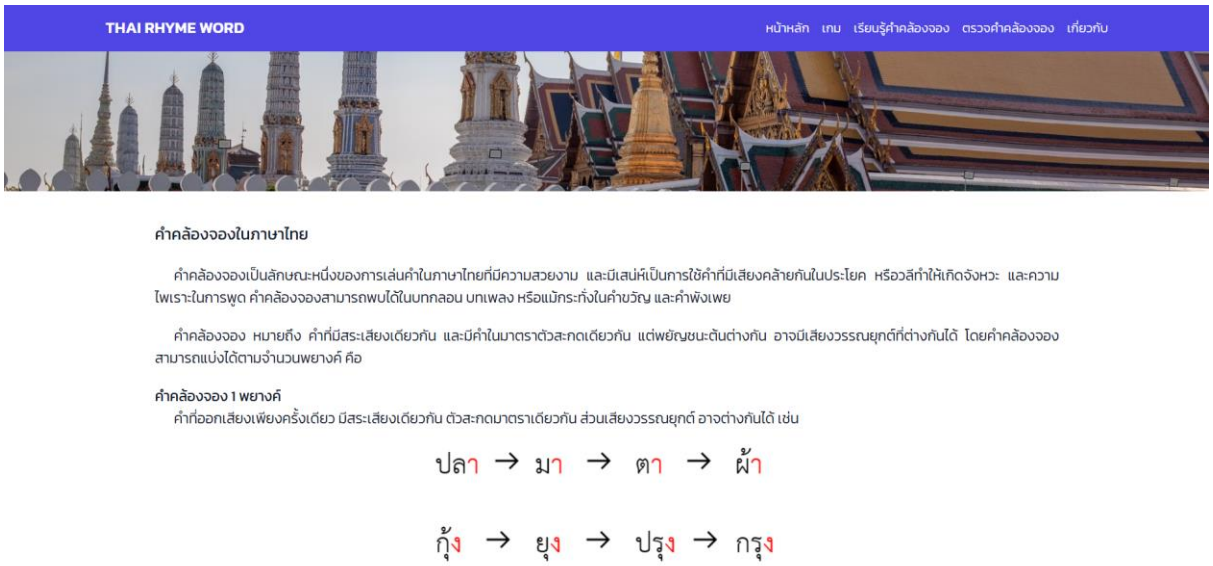
รูปผนวกที่ ก.6 หน้าเกมต่อคำคล้องจองขณะเล่นเกม



รูปผนวกที่ ก.7 หน้าแสดงผลลัพธ์หลังจบเกมต่อคำคล้องจอง

ก.3 หน้าเรียนรู้คำคล้องจอง

หน้าการเรียนรู้คำคล้องจองเป็นหน้าที่ใช้ในการสอนเรื่องคำคล้องจองของภาษาไทย โดยหน้าการเรียนรู้นี้ออกแบบมาเพื่อช่วยให้ผู้ใช้งานเข้าใจหลักการและวิธีการใช้คำคล้องจองในภาษาไทยได้ง่ายขึ้น



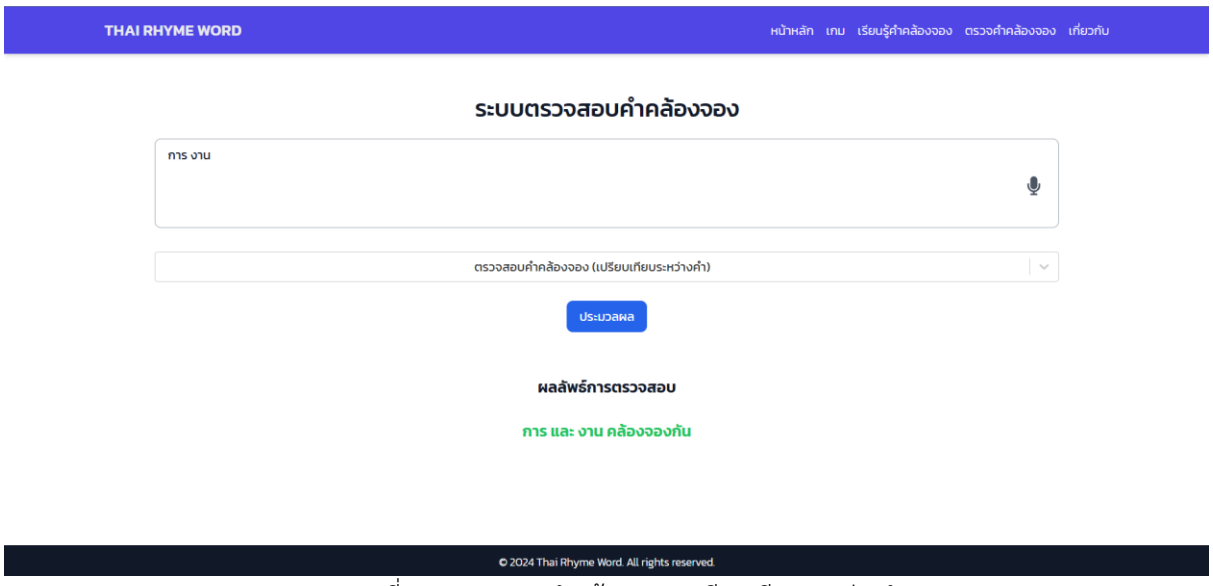
รูปผนวกที่ ก.8 หน้าสื่อการสอนของเว็บไซต์

ก.4 หน้าตรวจคำคล้องจอง

ในหน้านี้จะเป็นหน้าที่ใช้สำหรับการตรวจสอบคำคล้องจองโดยการพิมพ์คำ ข้อความลงในกล่องข้อความ และเลือกประเภทการตรวจสอบจากเมนูดรอปดาวน์ที่มีให้ โดยสามารถเลือกได้ว่าต้องการตรวจสอบแบบไหน ดังนี้

ก.4.1 เปรียบเทียบคำคล้องจอง

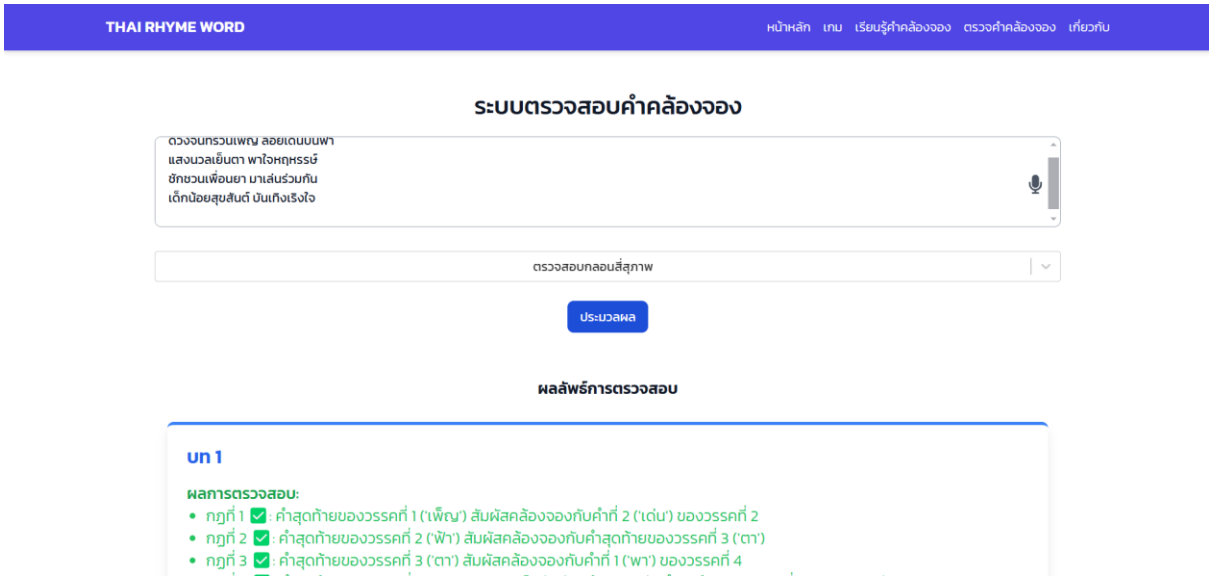
สำหรับการใช้งานการตรวจสอบคำคล้องจองแบบเปรียบเทียบระหว่างคำ ผู้ใช้งานต้องทำการป้อนคำที่ต้องการเปรียบเทียบโดยการพิมพ์คำแรก ตามด้วยเว้นวรรคหลังจากนั้นพิมพ์คำที่สอง จากนั้นกดปุ่มประมวลผล ระบบจะทำการแสดงผลลัพธ์การตรวจสอบออกมาว่าทั้งสองคำคล้องจองกันหรือไม่



รูปผนวกที่ ก.9 ตรวจสอบคำคล้องจอง (เปรียบเทียบระหว่างคำ)

ก.4.2 ตรวจสอบกลอนสี่

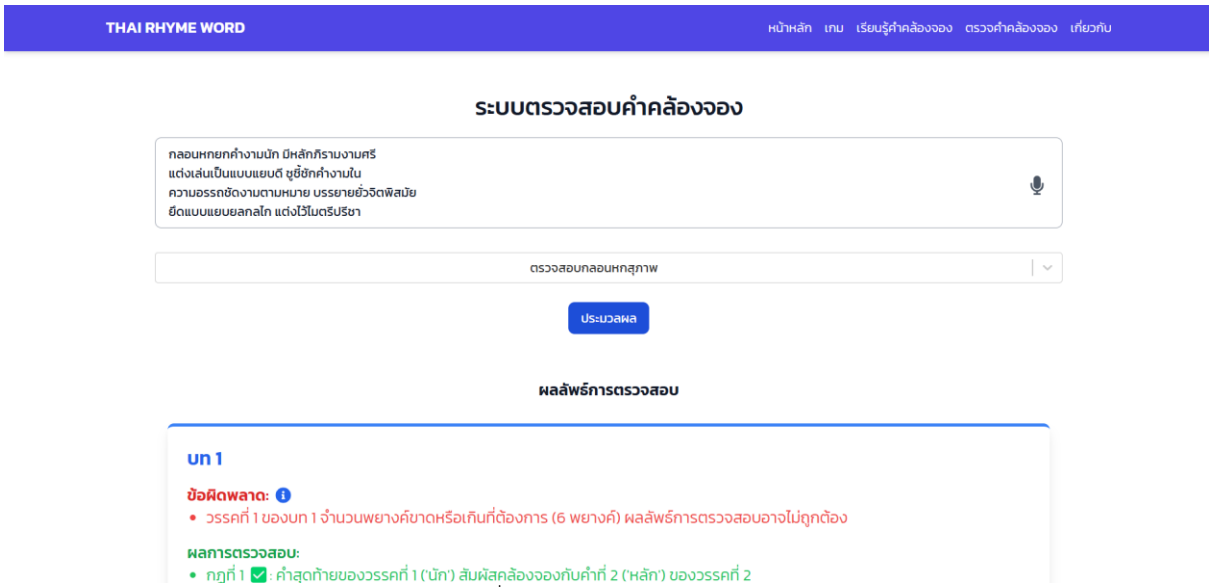
สำหรับการใช้งานการตรวจสอบคำคล้องจองแบบตรวจสอบกลอนสี่ ผู้ใช้งานต้องทำการป้อนกลอนสี่ลงไป จากนั้นกดปุ่มประมวลผล ระบบจะทำการแสดงผลลัพธ์การตรวจสอบกลอนสี่ออกมา



รูปผนวกที่ ก.10 ตรวจสอบกลอนสี่สุภาพ

ก.4.3 ตรวจสอบกลอนหก

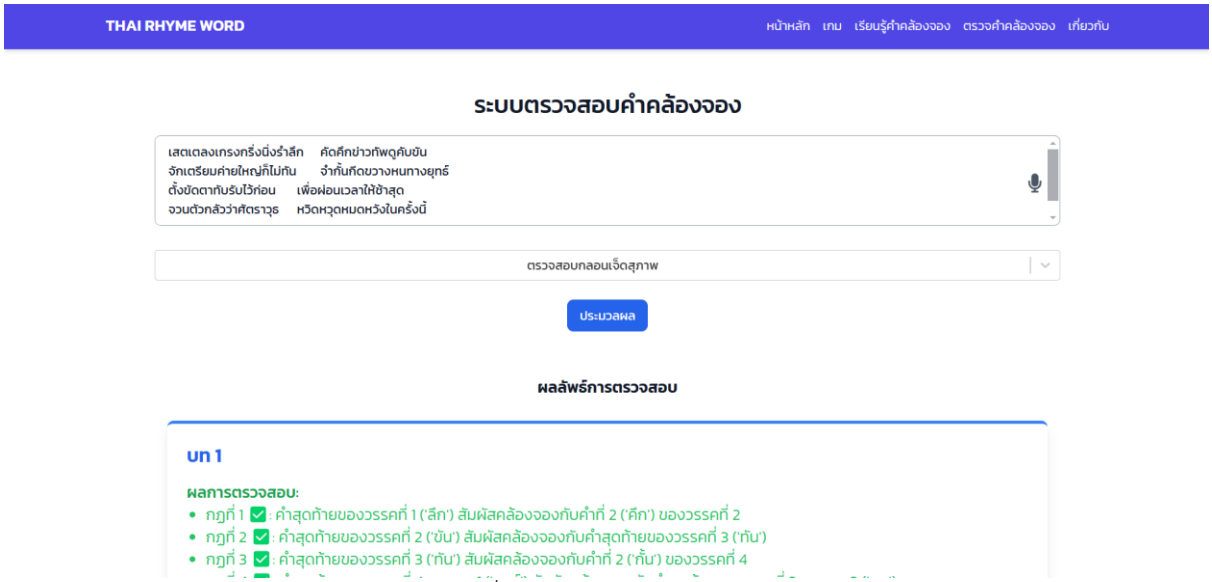
สำหรับการใช้งานการตรวจสอบคำคล้องจองแบบตรวจสอบกลอนหก ผู้ใช้งานต้องทำการป้อนกลอนหกลงไป จากนั้นกดปุ่มประมวลผล ระบบจะทำการแสดงผลลัพธ์การตรวจสอบกลอนหกออกมา



รูปผนวกที่ ก.11 ตรวจสอบกลอนหกสุภาพ

ก.4.4 ตรวจสอบกลอนเจ็ด

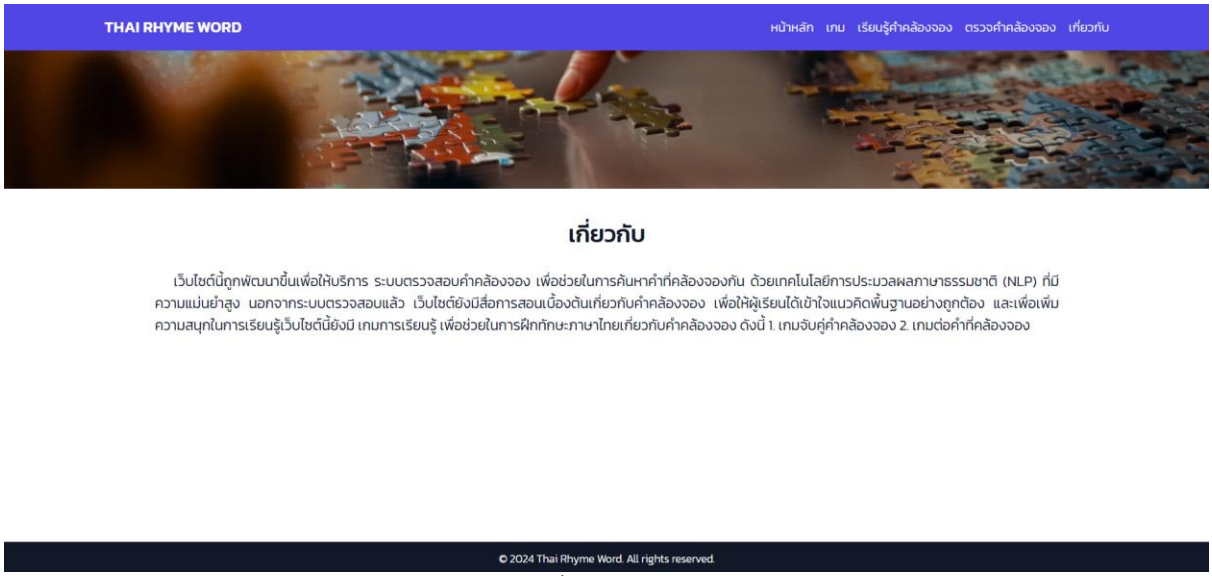
สำหรับการใช้งานการตรวจสอบคำคล้องจองแบบตรวจสอบกลอนเจ็ด ผู้ใช้งานต้องทำการป้อนกลอนเจ็ดลงไป จากนั้นกดปุ่มประมวลผล ระบบจะทำการแสดงผลลัพธ์การตรวจสอบกลอนเจ็ดออกมา



รูปผนวกที่ ก.12 ตรวจสอบกลอนเจ็ดสุภาพ

ก.5 หน้า About

สำหรับหน้า About จะใช้ในการอธิบายข้อมูลที่เกี่ยวกับเว็บแอปพลิเคชันเพื่อให้ผู้ใช้งานได้เข้าใจถึงแนวคิดและเป้าหมายของระบบได้อย่างชัดเจน



รูปผนวกที่ ก.13 หน้า About